

UNIDAD N1 – INGENIERÍA DE SOFTWARE

SOFTWARE

El software es un conjunto de programas en conjunto con toda la documentación para desarrollar y mantener los programas ejecutables que se entregan al cliente, como así también las herramientas utilizadas para la construcción de este.

El software es como, la información o conocimiento que se presenta en distintos niveles de abstracción, desde los requerimientos del sistema (abstracto), hasta el código que ejecuta el mismo (detallado).

INGENIERÍA DE SOFTWARE

La ingeniería de software es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación. Busca apoyar el desarrollo de software profesional en lugar de la programación individual.

Es una disciplina de la ingeniería que se preocupa por todos los aspectos de la producción del software; desde las primeras etapas de la especificación hasta el mantenimiento del sistema después de que se pone en operación.

Parnas [1987] definió a la ingeniería de software como “Multi-person construction of multi-version software”.

Los ingenieros aplican teorías, métodos y herramientas de la manera más conveniente siempre tratando de descubrir soluciones a los problemas, teniendo en cuenta que saben trabajar con restricciones financieras y organizacionales, por lo que buscan soluciones considerando estas restricciones.

La ingeniería de software nace tras la crisis del software, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de costos, productividad y la calidad de un software, debido principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollado.

ESTADO ACTUAL Y ANTECEDENTES – LA CRISIS DEL SOFTWARE

El término crisis del software hace referencia a un conjunto de hechos relativos al software, planteados en la conferencia de OTAN en 1968 por Friedrich L Bauer, quien recalcó la dificultad para generar software libre de defectos, fácilmente comprensibles y que sean verificables.

Refiere al conjunto de dificultades o errores en la planificación, estimación de costos, productividad y calidad de un software.

Causas:

1. La introducción de nuevas tecnologías de hardware que generaron la posibilidad de desarrollar software más complejo y de mayor tamaño. El salto en el hardware no fue acompañado por un salto en el desarrollo del software.
2. La complejidad que supone la tarea de programar.
3. Los cambios a lo que tiene que ser sometido un software para ser continuamente adaptado a las necesidades de los usuarios.

Los enfoques individuales al desarrollo de programas no escalaban hacia los grandes y complejos sistemas de software.

√ no eran confiables, costaban más de lo esperado y se distribuían con demora.

Entre 1970 y 1980 se desarrolló una variedad de nuevas técnicas y métodos de ingeniería de software que permitieron perfeccionar herramientas y notación estándar.

Antecedentes del software: anteriormente el hardware evolucionaba rápidamente y el software se desarrollaba de manera artesanal, habiendo algunos problemas como los sobrecostos, retrasos y el mantenimiento.

En la actualidad varios de estos problemas se han resuelto, pero siguen apareciendo nuevos relacionados a la complejidad, seguridad, costos, rapidez, etc.

DISCIPLINAS DE LA INGENIERÍA DE SOFTWARE

a) DISCIPLINAS TÉCNICAS: Ayudan a construir el producto

- * **Requerimientos:** se identifican los actores que interactúan con el sistema y se desarrollan casos de uso para modelar los requerimientos del sistema.
- * **Análisis y Diseño:** se crea y documenta un modelo de diseño utilizando modelos arquitectónicos, de componentes, de objetos y de secuencia.
- * **Construcción o Implementación:** se implementan y estructuran los componentes del sistema en subsistemas de implementación. La generación automática de código a partir de modelos de diseño ayuda a acelerar este proceso.
- * **Prueba:** las pruebas son un proceso iterativo que se realiza en conjunto con la implementación. Las pruebas del sistema siguen al completar la implementación.
- * **Despliegue:** Se crea la liberación de un producto, se distribuye a los usuarios y se instala en su lugar de trabajo.

b) DISCIPLINAS DE GESTIÓN: se centran en la organización, planificación, supervisión y control del proyecto de software.

- * **Planificación de Proyecto:** proporcionar un marco conceptual que permita hacer estimaciones razonables de recursos, costo y calendario. Antes de que el proyecto pueda comenzar, el equipo debe estimar el trabajo que va a realizar, los recursos que se requerirán y el tiempo que transcurrirá de principio a fin.
- * **Monitoreo y Control de Proyectos:** busca supervisar continuamente el progreso del proyecto en relación con el plan establecido, con el fin de detectar desviaciones a tiempo y aplicar acciones correctivas y preventivas que aseguren que el proyecto cumpla con sus objetivos.

c) DISCIPLINAS DE SOPORTE: son disciplinas transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software.

- * **Gestión de Configuración de Software:** identificar, organizar y controlar todos los elementos que forman parte del software a lo largo de su ciclo de vida. Implica llevar un control de las versiones del código fuente, documentación y artefactos, asegurar que los cambios se registren, se justifiquen y se aprueben antes de aplicarse.
- * **Aseguramiento de Calidad:** conjunto de actividades sistemáticas y planificadas orientadas a garantizar que el software cumpla con los estándares de calidad definidos. Definir buenas prácticas, realizar revisiones técnicas, aplicar normas de calidad y verificar que se cumplan los requisitos de cliente y del negocio.
- * **Métricas:** medidas cuantitativas que permiten evaluar, monitorear y mejorar tanto el proceso de desarrollo como el producto de software. Pueden ser métricas del proceso, del producto o del proyecto.

GRANDES PROYECTOS DE SOFTWARE FALLIDOS Y EXITOSOS

Amazon

El 14 de diciembre de 2014, numerosas personas se encontraron con una generosidad nunca vista: todos los productos de Amazon tenían una importante bajada de precio. Un teléfono o una aspiradora nueva costaban solo 0.01 libras, y todos los usuarios empezaron a hacer pedidos de millones de artículos.

La caída se debía a un fallo de software y había que solucionarlo antes de que las empresas se enfrentaran a una quiebra total. La venta tuvo lugar durante solo un día, pero el resultado fue devastador: algunas pequeñas empresas sufrieron pérdidas de hasta 100.000 libras.

La razón principal de este fracaso fue el funcionamiento del software de terceros. Repricer Express estaba utilizando precios dinámicos para asegurarse que los usuarios están comprando los artículos más baratos entre el alcance elegido. Algo salió mal y todos los precios alcanzaron su mínimo. Repricer se negó a asumir la responsabilidad de sus errores y no pagó por el trastorno.

Airbnb – Transformación del turismo

Airbnb utilizó el desarrollo de software para revolucionar la industria de la hospitalidad. Permitió la conexión entre anfitriones y viajeros de manera sencilla y transparente, implementando filtros avanzados para encontrar alojamientos adaptados a las preferencias del usuario y un sistema de pagos seguro y eficiente.

El éxito de Airbnb demuestra cómo un software bien diseñado puede crear modelos de negocio completamente nuevos.

Motivos de productos de software exitosos

- Involucramiento del usuario 15.9%.
- Apoyo de la Gerencia 13%.
- Enunciado claro de los requerimientos 9.6%.
- Planeamiento adecuado 8.2%.
- Expectativas realistas 7.7%.
- Hitos intermedios 7.7%.
- RRHH competentes 7.2%.

Causas de productos de software fallidos

- Requerimientos incompletos 13.1%.
- Falta de involucramiento del usuario 12.4%.
- Falta de recursos 10.6%.
- Expectativas poco realistas 9.3%.
- Falta de apoyo de la Gerencia 8.7%.
- Requerimientos cambiantes 8.1%.

CICLOS DE VIDA – MODELOS DE PROCESOS

Ciclos de vida

Los ciclos de vida son una serie de pasos a través de los cuales un producto o un proyecto progresa en el tiempo.

Son modelos genéricos de los procesos de software.

Es una representación simplificada de un proceso, el cuál define elementos del proceso y el flujo del proceso, que especifica la relación y el orden de dichos elementos. Es una representación de un proceso, el cual grafica una descripción del mismo desde una perspectiva particular.

Son abstracciones del proceso que se usan para explicar los distintos enfoques del desarrollo de software.

Los modelos de proceso pueden incluir las mismas actividades, pero cada uno, desde una perspectiva particular, pondrá más énfasis en algunas respecto a otras y definirán flujos de procesos diferentes.

Los Productos tienen ciclos de vida distinto al de los Proyectos → mientras transcurre el ciclo de vida del producto puede haber “n” ciclos de vida de proyectos que van a producir otras versiones del producto.

Características

- Especifican las fases del proceso y el orden en el cual se llevan a cabo.
- Es una guía para la administración del proyecto ya que indica el progreso a través de hitos.
- Los modelos de ciclos de vida se han vuelto necesarios debido a que los sistemas son más complejos por el aumento de funcionalidad y la mayor variedad de usuarios.
- Son independientes de los procedimientos de cada actividad del ciclo de vida.

Clasificación de los Ciclos de vida

No son mutuamente excluyentes y con frecuencia se usan en conjunto, sobre todo para sistemas complejos y de gran envergadura.

Secuenciales (cascada-puro)

Este modelo dispone las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada. El avance entre las fases se da tras una revisión final de cada una de estas para determinar si el software está listo para avanzar.

Toma las actividades fundamentales del proceso: especificación, desarrollo, validación y evolución, y los representa como fases separadas del proceso: especificación de requerimientos, diseño de software, implementación, pruebas, etc.

Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que [estos se congelan al final de la etapa de toma de requerimientos.

¿Cuándo elegirlo?

- Ayuda a minimizar los gastos generales de planificación porque se puede hacer toda la planificación por adelantado. No proporciona resultados tangibles en el desarrollo de software hasta el final del ciclo de vida, pero la documentación que se genera a lo largo de este proporciona indicaciones significativas de progreso a lo largo del ciclo de vida.
- Es más sencillo utilizar un método de gestión común a lo largo de todo el proyecto.
- Funciona bien para productos en los que se tiene una definición clara y estable, y cuando se trabaja con metodologías técnicas claras y no es probable un cambio drástico durante el desarrollo.

Desventajas

- Los proyectos raramente siguen un flujo secuencial.
- El modelo secuencial exige que los requerimientos sean completamente explícitos desde un principio y no suele suceder.
- No puede lidiar con la incertidumbre.
- El cliente obtiene una versión funcional del producto en etapas avanzadas del proyecto.
- Encontrar un defecto implica un rediseño en la solución, ya que está prácticamente todo hecho.
- Dependencia entre equipos de trabajo. Un equipo no puede comenzar hasta que el otro no haya finalizado.
- Desarrollo dirigido por un plan, y cada etapa genera documentación para realizar un monitoreo constante contra el plan, esta documentación puede llegar a ser muy burocrática y excesiva.

Iterativos (iterativo e incremental)

Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades.

Cada iteración produce un incremento de software funcional potencialmente entregable. Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente

Este enfoque vincula las actividades de especificación, desarrollo y validación. El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior.

Utilizado en metodologías ágiles.

Ventajas frente al modelo secuencial

- Se reduce el costo de adaptar los requerimientos cambiantes del cliente y el retrabajo es menor ya que en cada incremento se obtiene una retroalimentación que permite adaptar el modelo de las necesidades del cliente.
- El cliente es parte del proceso de desarrollo.
- Es posible que sea más rápida la entrega e implicación de software útil al cliente, aún si no se ha incluido toda la funcionalidad. Los clientes tienen la posibilidad de usar y ganar valor del software más temprano.
- Muy útil para sistemas de requerimientos cambiantes.
- La especificación, desarrollo y validación están entrelazadas en lugar de separadas y aisladas.

Desventajas

- Se invisibiliza el proceso.
- Alto costo de documentar cada incremento.
- La estructura del software tiende a degradarse frente a muchos incrementos.

Recursivos (espiral)

Se inicia con algo en forma completa, como una subrutina que se llama a sí misma e inicia nuevamente. Se presenta un prototipo que va mejorando con cada vuelta.

El modelo recursivo es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente. El modelo en espiral es un modelo recursivo que consta de 4 etapas recursivas:

1. Determinar objetivos.
2. Identificar y resolver riesgos.
3. Desarrollar y probar.
4. Planificar siguiente iteración.

Ventajas

- Se generan productos independientes de la implementación, que pueden ser reusables en sistemas de características similares.
- Permite explorar requisitos poco claros desde el inicio.

Desventajas

- Puede ser más costoso en tiempo y dinero readaptarlos para reutilizarlos para diferentes proyectos.
- La tecnología puede ser obsoleta.
- Puede carecer de mantenimiento o documentación, lo cual dificulta mucho las tareas.
- Se genera una versión del producto recién al final.

Modelos de Ciclos de vida

Modelo de Cascada

Proceso dirigido por un plan: en principio, usted debe planear y programar todas las actividades del proceso, antes de comenzar a trabajar con ellas. Se sigue una secuencia de pasos ordenada y se realiza revisión al final de cada fase. Es conducida por la documentación con fases que se superponen.

- Permite encontrar errores en etapas tempranas.
- Hay exceso de documentación y falta de resultados hasta el final.
- Se utiliza cuando la definición del producto es estable o los requerimientos de calidad dominan a los de costo y tiempo.

Modelo en Espiral

Busca minimizar los riesgos haciendo un análisis de riesgos y buscando alternativas al iniciar cada fase. Al final de cada fase planifica la siguiente y se evalúa si se sigue adelante.

- Permite evaluar la factibilidad de un nuevo producto.
- En proyectos grandes la identificación de los riesgos puede costar más que el desarrollo.
- Es un modelo adecuado para proyectos con ciclos de vida largos, que puede utilizar equipos diferentes en cada ciclo y muestra el progreso al fin de cada ciclo.

Modelo Evolutivo

Son iterativos. Se caracterizan por la manera en la que permiten desarrollar versiones cada vez más completas del software. Se fija el tiempo o el alcance mientras lo otro se va modificando. Es útil cuando los requerimientos no son claros y se realizan entregas tempranas al cliente.

- El problema es que es un modelo recursivo que repite todas las tareas y depende de que los diseñadores desarrollen un sistema fácil de modificar.

La Administración de Proyectos de Software dependiendo en Ciclo de vida

La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos. Por ejemplo, si se realiza un análisis de riesgo y se determina que los requerimientos pueden variar mucho a lo largo del desarrollo, se debería elegir un ciclo de vida en el cual el cambio sea permisible, como los iterativos. La administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida.

Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección, ya que no es lo mismo gestionar un proyecto que se desarrolla de manera iterativa, que uno que se desarrolla de manera secuencial o en cascada.

La **Administración de Proyectos** implica la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto. Administrar un proyecto correctamente implica tener el trabajo hecho en tiempo, con el presupuesto acordado, habiendo satisfecho los requerimientos.

Algo indispensable en la Administración de Proyectos es el **Plan del Proyecto**, es el documento en el cual se especifica el alcance del proyecto, el calendario en base al cual se va a llevar a cabo, las actividades y tareas que se pondrán en marcha para lograr el objetivo y los responsables que llevarán a cabo cada una de las tareas descriptas.

La planificación de un proyecto implica, entre otras, **definir el proceso y ciclo de vida** del proyecto.

El ciclo de vida influye directamente en la administración del proyecto ya que define la forma en que se planifica (secuencial, iterativa, recursiva), determina los puntos de control y cómo medir el avance, condiciona la gestión de riesgos e impacta en los costos, tiempos y calidad del producto final.

PROCESOS DE DESARROLLO - EMPÍRICOS vs DEFINIDOS

Un proceso es una secuencia de pasos ejecutados para un propósito dado.

Un proceso de software es un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

Sus actividades son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba.

El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.

Procesos Definidos

Los procesos definidos son considerados deterministas. Un proceso definido asume que podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados (receta de cocina). Es un proceso estructurado.

- Inspirados en las líneas de producción.
- La administración y el control provienen de la predictibilidad del proceso.
- El objetivo es lograr una repetibilidad, para poder obtener una previsibilidad de esfuerzo, riesgos, costos, etc.
- Se utilizan en actividades de software bien conocidas y estructuradas como compilación o reportes.
- Pueden usar cualquier ciclo de vida.

Procesos Empíricos

Los procesos empíricos se conforman en base a la experiencia interna y externa de las personas que intervienen en un contexto particular.

Son procesos complicados con variables cambiantes, al repetir el proceso se pueden obtener distintos resultados. Dado que el factor diferencial es la experiencia sensible, en estos procesos se pueden obtener diferentes resultados dependiendo del contexto en el cual se apliquen.

- Se basa en ciclos cortos de inspección y adaptación (retroalimentación).
- La administración y control vienen a través de inspecciones frecuentes y adaptaciones.
- Se usan en actividades donde el conocimiento es incompleto o evoluciona con el tiempo.
- La misma entrada puede generar distintas salidas.
- Solo se usa en procesos iterativos.

La experiencia que se obtiene en cada uno de los procesos no es extrapolable, es decir, que solo funciona en ese mismo proyecto, y se necesita adaptar los procesos a cada contexto especial.

Los conocimientos obtenidos en el proceso se deben hacer explícitos al grupo de desarrollo, y convertir las habilidades individuales en buenas prácticas del trabajo en grupo (compartir conocimiento). Haciendo referencia al pilar de la transparencia.

Está basado en pilares:

1. **Transparencia:** nos permite crecer como equipo y transformar el conocimiento implícito en explícito, es nuestro código no mío. Todos tienen acceso a la información.
2. **Inspección:** permite detectar problemas y varianzas, es el momento de emprender y ganar experiencia.
3. **Adaptación:** adapto según la inspección. Es la mejora continua.

Pasos del patrón de conocimiento de patrones empíricos:

1. **Asumir:** se parte de supuestos.
2. **Construir:** se prueba la hipótesis.
3. **Retroalimentación:** se recogen datos y opiniones del resultado.
4. **Revisar:** se analiza resultados frente a lo esperado.
5. **Adaptar:** ajustar según lo aprendido.

CICLOS DE VIDA Y PROCESOS DE DESARROLLO DE SOFTWARE

Proceso y Proceso de Software

Un **proceso** es una secuencia de pasos ejecutados para un propósito dado, mientras que un **proceso de software** es un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

- Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse, pero deben incluir: productos, roles, responsabilidades y condiciones.
- El proceso debe ser explícitamente modelado si va a ser administrado.
- Un **proceso de desarrollo** se puede aplicar a varios **ciclos de vida**. La elección del ciclo de vida depende de la estrategia y este debe permitir seleccionar los artefactos a producir, definir actividades y roles, y modelar conceptos.
- Las personas utilizan herramientas y equipos para llevar a cabo los procedimientos que componen el proceso de desarrollo.

Actividades fundamentales

- Especificación de software: clientes junto con ingenieros definen el software a producir y sus restricciones.
- Desarrollo: diseño y programación de software.
- Validación: verificación para asegurar que es lo que el cliente quiere.
- Evolución: donde se modifica el software para reflejar los requerimientos cambiantes del cliente y mercado.

Ciclos de vida

Son modelos genéricos de los procesos de software, son abstracciones del proceso que se usan para explicar los distintos enfoques del desarrollo de software.

Un **ciclo de vida** de software es una **representación de un proceso**, el cual grafica una descripción del mismo desde una perspectiva particular.

El **proceso de desarrollo de software** es el **qué**, conjunto de actividades y prácticas que permiten desarrollar software.

El **ciclo de vida** es el **cómo**, una abstracción que organiza esas actividades en fases y orden de ejecución.

VENTAJAS Y DESVENTAJAS DE LOS CICLOS DE VIDA

Ciclo de Vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	• Alta certeza (baja incertidumbre). • La organización tiene una forma tradicional de trabajar. • No se pueden realizar entregas parciales del software (se debe entregar todo junto). • Eliminar riesgos de planificación.
Iterativo/incremental	Definidos o Empíricos	• Existe incertidumbre. • Volatilidad de requerimientos. • Posibilidad de entregas parciales. • Uso en etapas tempranas del producto.
Recursoivo	Definidos	• Mucho control de riesgos en cada iteración. • No se pueden realizar entregas parciales.

CRITERIOS DE ELECCIÓN DE LOS CICLOS DE VIDA

Criterios para la selección de un modelo de proceso

Es necesario evaluar los niveles de riesgo, la claridad de requerimientos, los conocimientos técnicos con los que cuenta el equipo de desarrollo, los plazos que se tienen para ejecutar el proyecto y la posibilidad de poder validar los resultados del trabajo con el cliente, también el equipo de trabajo.

Secuencial: si los requerimientos son claros y exhaustivos, en un contexto de baja incertidumbre en el que el cliente no requiere obtener el producto rápidamente.

Iterativo e incremental: si la situación anterior no se presenta y el cliente desea obtener resultados lo antes posible, con las funciones principales del sistema.

Recursivo: si el objetivo es disminuir los riesgos presentes en el desarrollo de sistemas grandes y complejos.

Por lo tanto, algunos de los criterios son: estabilidad de requisitos, incertidumbre técnica, presupuesto, etc.

COMPONENTES DE UN PROYECTO DE SOFTWARE

Proyecto de Software

Un proyecto es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único.
Un proyecto es una unidad de gestión de trabajo.

Es un medio por el cual se pueden organizar los recursos que se necesitan y las personas, para poder obtener como resultado un producto o servicio. El proyecto necesita de personas y recursos, y de un método o indicación de cómo hacer el trabajo para cumplir con el objetivo.

El proyecto instancia al proceso y lo adapta a sus necesidades.

Características de los Proyectos

1. **El resultado que se obtiene es único:** todos los proyectos por más similares que sean tienen características que los hacen únicos. El resultado de un proyecto será único e irrepetible para cualquier otro proyecto.
2. **Orientado a objetivos:** los proyectos están dirigidos a obtener resultados y ello se refleja a través de objetos. Los objetivos guían el proyecto, no deben ser ambiguos y debe ser alcanzable. Los objetivos deben ser:
 - No ambiguos: deben ser claros para guiar el proyecto.
 - Medibles: medir el objetivo para determinar el avance del proyecto.
 - Alcanzables: deben ser factibles de realizarse por el equipo.
3. **Duración limitada:** los proyectos son temporales, cuando se alcanza el/los objetivo/s, el proyecto termina. Tiene una fecha de inicio y fin.
4. **Tareas interrelacionadas basadas en esfuerzos y recursos:** se debe a la complejidad sistémica de los problemas. Para alcanzar el objetivo, teniendo los recursos disponibles, se debe dividir el trabajo en tareas. Esas tareas están relacionadas.

Administración de Proyectos

Administración de proyectos es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto.

Administración de proyectos es la organización y coordinación de personas y recursos que permitan desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando la entrega de un producto o servicio de alta calidad.

Administrar un proyecto correctamente implica tener el trabajo hecho en tiempo, con el presupuesto acordado, habiendo satisfecho los requerimientos.

Esta disciplina tiene dos grandes actividades: planificación y monitoreo y control.

La administración de proyectos incluye:

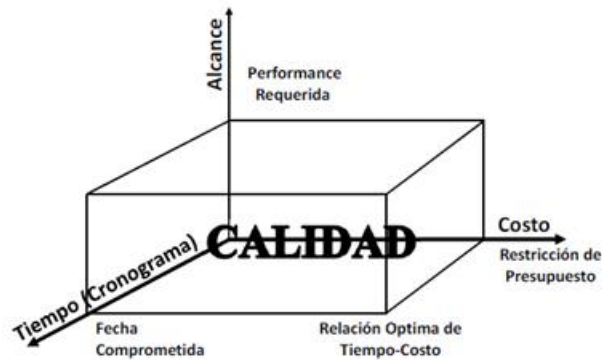
- Identificar requerimientos.
- Establecer objetivos claros y alcanzables.
- Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).

Una buena gestión no puede garantizar el éxito del proyecto. Sin embargo, una mala gestión usualmente implica una falla en el proyecto. Un proyecto planificado puede fracasar también, lo importante de la planificación es el acto de planificar, no el resultado.

Metas:

- Entregar software al cliente en tiempo acordado.
- Mantener los costos dentro del presupuesto general.
- Entregar software que cumpla con las expectativas del cliente.
- Mantener un equipo de desarrollo óptimo y con buen funcionamiento.

Triple Restricción o Triángulo de Hierro



Para lograr entregar ese producto o servicio se necesita tiempo, recursos (y personas) que generan costos.

Alcance: son los requerimientos del proyecto, es decir, los límites o el ámbito sobre el cual se va a mover el mismo. Es la variable que primero se negocia con el cliente. Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto.

Tiempo: calendario, cuáles serán las fechas especificadas para realizar las entregas que determinarán el alcance del proyecto.

Costo: son los recursos que se verán implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, etc.

El concepto de la triple restricción hace referencia a que no es posible fijar de forma arbitraria a las 3 variables, sino que será posible fijar 2 y la tercera se ajustará a lo definido.

Debemos poder manejar al menos 2 de estos, el alcance del sistema siempre maneja la restricción del alcance y con esto podremos definir tiempo y costo.

El balance de estos tres factores afecta directamente la calidad del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto requerido, el servicio o resultado, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado.

Es responsabilidad del **Líder de proyecto** balancear estos tres objetivos, que a menudo compiten entre ellos y por lo general casi nunca se cumplen simultáneamente.

En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo. Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben aumentar, para lograr abordar un proyecto más grande.

El problema en la aplicación de esta visión en **enfoques tradicionales** es que, al utilizar procesos definidos con ciclos de vida en cascada, los requerimientos se definen al comienzo del proyecto, por lo que cualquier modificación o cambio en el alcance del producto final, modifica el tiempo y presupuesto de este.

Rol del Líder del Proyecto

Para ser líder uno tiene que sentirse cómodo con los cambios y entender la organización.

El líder debe tener Hard Skills (conocimientos del producto, herramientas y técnicas) y los Soft Skills (capacidad de trabajar con gente, comunicación, liderazgo, creatividad, organización, motivación, empatía).

Un líder de proyecto debe tener las siguientes aptitudes: motivación, organización, innovación, empatía, comunicación y liderazgo.

También cuenta con habilidades duras: resolución de problemas, identidad administrativa, logro e influencia.

En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo. Es quien se ocupa de todas las actividades de gestión del proyecto. Es el máximo responsable de que el proyecto se termine en tiempo y forma.

Responsabilidades del líder de proyecto:

1. Definir el alcance del proyecto.
2. Identificar involucrados, recursos y presupuesto.
3. Detallar tareas, estimar tiempos y requerimientos.
4. Identificar y evaluar riesgos.
5. Preparar planes de contingencia.
6. Controlar hitos y participar en las revisiones de las fases del proyecto.
7. Administrar el proceso de control de cambios y producir reporte de status.

Equipo de Proyecto

Grupo de personas comprometidas en alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.

Características:

1. Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades. Tienen un sentido de mejora continua y aprendizaje mutuo.
2. Usualmente son un grupo pequeño, esto permite lograr una comunicación efectiva entre los miembros del equipo.
3. Trabajan en forma conjunta con espíritu de grupo, convirtiéndose en un equipo cohesivo donde prima la sinergia de este. Se debe lograr un equipo maduro y estable.
4. Sentido de responsabilidad como una unidad y se focaliza en el cumplimiento de las metas grupales.

Stakeholders

Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

- El patrocinador debe tener jerarquía en la empresa y garantizar recursos.
- En metodologías ágiles el líder del proyecto es el Scrum Master y el patrocinador el Product Owner.

Plan de Proyecto

El Plan de Proyecto es un artefacto de gestión que cumple la función de “hoja de ruta” de un proyecto.

Se documentan todas las decisiones que se toman a lo largo del desarrollo; en términos de: el proceso, la división de tareas a realizar, asignación de responsabilidades, equipos de trabajo, calendario de desarrollo, actividades de soporte (planes de contingencia, mecanismos para control de cambios, evaluar calidad y valorar riesgos).

El líder de proyecto debe poder obtener un plan de proyecto. Ese plan de proyecto es un mapa que define el “viaje” o cómo va a realizarse el proyecto. Es el artefacto resultante del proceso de planificación.

¿Qué es lo que hacemos? → Alcance

¿Cuándo lo hacemos? → Calendarización

¿Cómo lo hacemos? → Recursos

¿Quién lo va a hacer? → Asignación de Tareas

Este documento es de suma importancia que sea correctamente mantenido y actualizado de manera permanente, es decir “no está escrito en piedra” y tiene ciclos de cambios. Si el plan de proyecto no se actualiza es muy probable que el proyecto tienda al fracaso. Tiene que servirle al líder como guía y debe actualizarse en base a lo que pasa.

Responde a estas preguntas:

1. Definición del alcance del proyecto.
2. Definición de proceso y ciclo de vida.
3. Estimación.
4. Gestión de riesgos.
5. Asignación de recursos.
6. Programación de proyectos.
7. Definición de métricas.
8. Definición de controles.

Planificación de un Proyecto

1. Objetivo del Proyecto

Qué vamos a hacer, el producto al que se quiere llegar.

- El objetivo debe ser claro para todo el equipo, debe haber una visión compartida en la que todos tengan la misma comprensión.
- Debe ser alcanzable → posible de lograr.
- Requerimientos funcionales y no funcionales.

2. Definir el Alcance

Es todo el trabajo y sólo el trabajo que es necesario realizar para alcanzar el objetivo.

- Todo el trabajo y solo el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas.
- Son las tareas que deben realizar para cumplir con el objetivo del proyecto y se guardan en el plan del proyecto.

El alcance de un producto y el del proyecto son diferentes. El alcance de un producto define qué funcionalidades debe realizar el software para satisfacer los requerimientos del cliente.

Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto, pero esto no necesariamente tiene que cambiar el alcance del producto. Por otra parte, el alcance de un producto sí condiciona los alcances de un proyecto. (Puede ocurrir que no siempre sea así).

3. Definir el Proceso y el Ciclo de Vida

Al definir el proceso que se va a utilizar, se responde al cómo se va a desarrollar el proyecto.

Se debe tener en cuenta el contexto del desarrollo y la complejidad del software a desarrollar.

El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo.

El ciclo de vida de un proyecto define:

1. Qué trabajo técnico debería realizarse en cada fase.
2. Quién debería estar involucrado en cada fase.
3. Cómo controlar y aprobar cada fase.
4. Cómo deben generarse los entregables.
5. Cómo revisar, verificar y validar el producto.

Considerar al máximo todo lo que hay que hacer para hacer software → describir lo máximo que se puede necesitar.

En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (si o si tiene que ser iterativo e incremental).

4. Estimación

Se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto.

En un enfoque tradicional, esta estimación de valores es realizada únicamente por el líder de proyecto. En la metodología tradicional, existe un orden definido y recomendado para realizar las estimaciones. Se estima:

- **Tamaño del producto:** el aumento del tamaño trae consigo un aumento de complejidad del software.
- **Esfuerzo:** cantidad de horas persona lineales haciendo una actividad por vez, dentro del desarrollo del producto.
- **Calendario del proyecto:** cuándo se van a realizar las tareas definidas. Proporcional a su crecimiento.
- **Costos:** depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo, es lo último en estimar.
- Recursos críticos.

5. Gestión de Riesgos

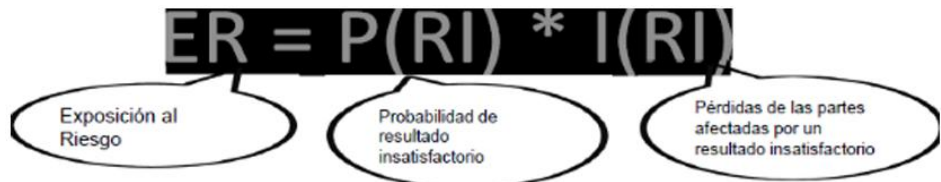
Un riesgo es la probabilidad de que un evento no deseado ocurra en un proyecto que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.

Es de suma importancia identificar y definir los riesgos a los que está atado un proyecto, para así poder analizarlos, y realizar una cuantificación de estos, teniendo en cuenta la exposición del proyecto ante los mismos, para así definir a cuáles se debe atender y sobre cuáles se debe hacer un seguimiento.

Clasificación de riesgos:

- **Riesgos del proyecto:** afectan el calendario o los recursos del proyecto. Ejemplo: renuncia de un diseñador experimentado.
- **Riesgos del producto:** afectan la calidad o el rendimiento del software que se está desarrollando. Ejemplo: la falla de un componente adquirido al no desempeñarse como se esperaba.
- **Riesgos del negocio:** afectan a la organización que desarrolla o suministra el software. Ejemplo: un competidor que introduce un nuevo producto.

Para medir los riesgos se utiliza lo que se conoce como exposición al riesgo (probabilidad * impacto), la cual se obtiene como el producto entre la probabilidad de ocurrencia de la amenaza por el impacto que generaría si realmente ocurriera. Este valor permite organizar y gestionar los riesgos, y se debe hacer foco en reducirla.



Ante los riesgos identificados, es posible tomar 3 actitudes:

- Negación: hacer de cuenta que no existen.
- Gestión Reactiva: esperar a que el riesgo ocurra y recién ahí ver qué hacer.
- Gestión Proactiva: ir trabajando sobre el riesgo generando planes de mitigación y de contingencia.

Riesgos comunes:

- Cambio de requerimientos en el medio de un desarrollo.
- Abandono de integrantes del equipo de desarrollo.
- Falta de capacidad del equipo.
- El gran avance tecnológico, no tener claro el funcionamiento de las últimas tecnologías.

Pasos de la gestión de riesgos:

1. **Identificación de riesgos:** se identifican los riesgos que suponen una mayor amenaza al proceso de ingeniería de software, al software a desarrollar o a la organización a desarrollar.
2. **Análisis de riesgos:** para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto.
3. **Planeación del riesgo:** para cada riesgo analizado, se definen estrategias para manejarlos. Estas estrategias consisten en considerar acciones a tomar para minimizar o evitar el impacto sobre el proyecto como consecuencia de la amenaza que representa el riesgo.
4. **Monitorización del riesgo y control:** proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado.

6. Asignación de responsabilidades (o recursos)

La asignación de responsabilidades a personas, permite asignar roles a los integrantes del equipo de trabajo. En el entorno de procesos definidos, estos roles se ven definidos en el concepto de trabajadores.

Dentro de un mismo proyecto, una persona puede desenvolverse en más de un rol, ya que puede que el presupuesto del desarrollo de software no sea lo suficientemente grande para permitirse tener una persona distinta por rol, o bien, el proyecto no sea lo suficientemente grande o complejo para justificar dicha adición.

Para dejar explícitos los roles y quienes son las personas que los cumplen, generalmente se construye una tabla, la cual contiene la información de la persona, el rol y responsabilidades de esta en el contexto del proyecto.

7. Programación de proyectos (calendarización)

En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo. Se toma como base lo definido en la estimación del calendario realizada previamente, pero se refina al máximo detalle posible cada actividad.

Cuando se habla de detallar las tareas, se hace referencia a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quién la realiza, cuándo debe realizarse y cuánto esfuerzo debe llevar en forma teórica.

8. Métricas

Las métricas se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo.

Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio (producto, proyecto o proceso) en conjunto con la forma de calcularlas.

Las métricas deben ser representativas para el entorno, es decir, que representen un aumento en la información y benefician la practicidad del desarrollo y seguimiento.

A diferencia de las metodologías ágiles, acá se definen métricas que miden y exponen los avances. En cambio en ágil, el mayor indicador de avance de un producto, son los incrementos que se entregan en cada iteración.

Métricas básicas para un proyecto de software:

- Tamaño del producto.
- Esfuerzo, del proyecto y proceso.
- Calendario, del proyecto y proceso.
- Defectos del producto.

Clasificación:

- **Métrica Directa:** se puede obtener sin depender de ninguna otra métrica y cuya forma de medir es a través de un método de medición.
- **Métrica Indirecta:** proviene de una función de cálculo cuyos argumentos son otras métricas directas o indirectas.

Las métricas de software tienen 3 dominios: proceso, producto y proyecto. La consolidación de las métricas de proyecto crea métricas de proceso que son públicas para toda la organización.

9. Monitoreo y control

El control de proyectos se refiere a comparar el progreso con respecto al plan, con el objetivo de verificar si estamos bien encaminados y, de no ser así, generar medidas preventivas que nos permitan volver a lo esperado. Si ya se han detectado desviaciones deberán tomarse acciones correctivas.

El objetivo es conocer el estado del proyecto:

- **Proyecto bajo control:** se están alcanzando hitos del proyecto a tiempo con los recursos estimados y con un nivel de calidad esperado. Se cumplen estándares y planificación.
- **Proyecto fuera de control:** se deberá replanificar y renegociar el plan.

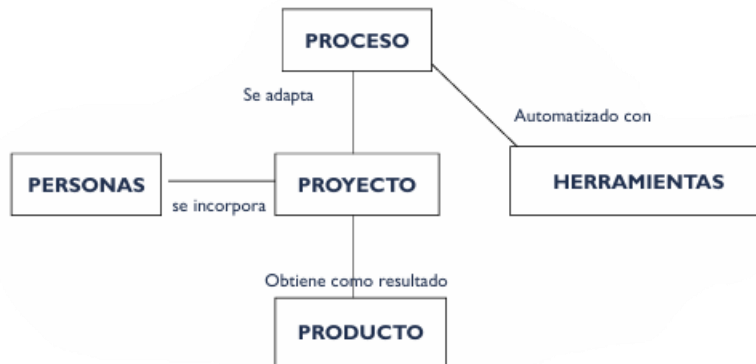
Consideraciones:

- Para el seguimiento se realizan reuniones semanales que pueden incluir encuentros con el cliente.
- Se puede incluir check-list de fin de fase, revisiones del plan, auditoría y reuniones post mortem.
- Sobre el gráfico de Gantt se puede estimar el porcentaje de tareas no finalizadas y que aún no han comenzado, debiendo haberlo hecho.
- Se realiza seguimiento o tracking para minimizar riesgos conocer y aceptar la realidad y tomar acciones correctivas.
- Seguimiento de proceso: se realiza un control por hitos o por tareas.
- Seguimiento de producto: se realiza un control sobre los entregables.

Factores de éxito de un proyecto:

- Monitoreo & feedback.
- Misión/objetivos claros.
- Comunicación.

VÍNCULO PROCESO-PROYECTO-PRODUCTO



Se dice que el proceso, automatizado con herramientas, se adapta al proyecto, al cual se incorporan personas, y de este se obtiene como resultado las personas.

En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito del producto para poder realizar las estimaciones pertinentes. Los objetivos identifican las metas globales del producto sin especificar cómo se van a lograr. El ámbito del producto define las funciones y comportamientos que caracterizan al producto.

Para poder obtener como resultado un producto o servicio, es necesario que el proyecto adopte un proceso de desarrollo para poder definir qué actividades, métodos, prácticas y transformaciones se utilizarán, mediante las cuales se va a obtener el software.

Dentro de cada proyecto de desarrollo de software, es necesario utilizar un proceso diferente dependiendo del tipo de producto que se desea desarrollar, ya que puede que la complejidad de este sea un determinante en la elección del proceso. Es por esto, que es necesario adaptar los procesos a cada uno de los proyectos en los cuales se trabaja, es decir, utilizar solo lo que verdaderamente hace falta para el desarrollo de este, ya que no existe un proceso ideal que sirva para cualquier tipo de proyectos.

El proceso es una definición teórica de lo que debería hacerse para tomar los requerimientos y transformarlos en un producto o servicio, guiados por el objetivo de lo que se quiera construir.

El proceso proporciona un marco conceptual en donde se establece un plan completo para el desarrollo de software. Define cómo se va a desarrollar el software, estableciendo un conjunto de actividades. Las disciplinas de gestión permiten que las actividades del marco conceptual se adapten a las características del proyecto de software y a los requerimientos del equipo.

Las disciplinas de soporte son independientes del marco conceptual y recubren al modelo del proceso, es decir, atraviesan todas las actividades del proceso de desarrollo.

Se debe definir el modelo de proceso a utilizar: secuencial, iterativo o recursivo. Luego el marco conceptual que incluye las actividades y formas de trabajo que deberán utilizar y seguir para lograr el desarrollo del producto/servicio deseado.

Las personas son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva. Si se adopta el “mejor proceso” para el desarrollo de un software, y además se destina mucho esfuerzo a planificar el proyecto de desarrollo, de nada servirá si no tenemos un equipo capacitado que reúna las habilidades y herramientas necesarias para el desarrollo del mismo.

En el desarrollo de un proyecto, se busca utilizar diversas herramientas que se adapten y permitan automatizar o facilitar las actividades que están definidas en el proceso adoptado. Por ejemplo, el uso de un software de diseño para realizar el análisis y diagramación de clases.

UNIDAD N2 – GESTIÓN LEAN-AGILE DE PRODUCTOS DE SOFTWARE

GESTIÓN DE PRODUCTO – AMBIENTES ÁGILES

Un producto de software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos. Cuando creamos un producto debemos enfocarnos en aquellos que es de valor para el cliente.

Las organizaciones que usan el software se enfocan en la parte superficial del software, aquello que consideran significativo y placentero. Usualmente no se fijan en si el producto es funcional o confiable.

Creamos productos para:

- Satisfacer a los clientes.
- Tener muchos usuarios logueados. Saber qué tan aceptado es el producto por la cantidad de usuarios que lo instalan.
- Para obtener mucho dinero.
- Realizar una gran visión, cambiar el mundo.

Evolución de los productos de software

La pirámide está dividida en niveles que representan diferentes etapas de madurez del producto, desde lo más básico hasta lo más valioso para el usuario.

Cuando vamos a desarrollar un software la mayoría de las veces se empieza a construir un producto sobre la base de lo funcional, que haga lo que tiene que hacer (tareas a realizar), luego si se quiere avanzar más, el software debe ser confiable donde los usuarios no corran peligro usando el producto y los resultados que este producto me da yo pueda depender de ellos sin ningún problema. Luego, la usabilidad tiene que ver con aspectos de la disciplina UX/UI, donde el usuario podrá lograr sus objetivos productivamente usando este producto, estando a gusto con su uso. Tiene que ser productivo.

Existe una línea en la pirámide que separa la usabilidad de la conveniencia. La mayoría de los procesos no se ubican por encima de esta línea, debido a que un software sea conveniente para el desempeño de tareas de ciertas personas (es decir, que el producto nos haga sentir que lo necesitamos porque nos produce un cambio, una facilidad para cumplir con un objetivo), y que estas no sean “esclavos” del software, limitándose a cumplir con las características de funcional, confiable y/o usable. Placentero y significativo para todo el mundo, algo que nos cambie la forma de usar algo o de vivir.

La parte inferior, focalizada en Output, donde las organizaciones se enfocan más en lo que el producto hace (características, funciones, tareas). La parte superior, focalizada en el Outcome, donde se busca generar valor emocional, contextual y duradero para los usuarios. No se trata solo de usar, sino de sentir que vale la pena usarlo.

Gestión de Productos

La gestión de productos es una disciplina que se encarga de planificar, desarrollar, lanzar y mejorar productos de software de manera que satisfagan las necesidades del usuario, sean viables técnicamente y rentables para el negocio. Esa debe ser la base, pero el producto debe proveer una experiencia que cumpla con todas las expectativas del usuario.

- Poder ver el software como un producto.
- No más software a medida:
 - la demanda es hacia productos que se venden muchas veces (productos enlatados).
 - sólo para organizaciones muy específicas o únicas.
- Productos prefabricados que se pueden personalizar.
 - se especifican configuraciones.
 - versiones que se puedan configurar.



Ciclo de vida de un Producto

El ciclo de vida de un producto describe cómo evoluciona un producto desde su creación hasta su desaparición del mercado.

1. *Fase de Introducción*

- * Nace la idea de crear un producto, es un momento crítico.
- * El producto es lanzado por primera vez al mercado, normalmente en forma de **MVP**.
- * El objetivo es validar si el producto resuelve un problema real, obtener feedback de usuarios tempranos y corregir errores iniciales y mejorar.

2. *Fase de Crecimiento*

- * El producto empieza a ganar aceptación en el mercado y las ventas aumentan rápidamente.
- * Se lanzan nuevas funciones del mismo producto, **MVF**, **MMF**.
- * El objetivo es escalar el producto (infraestructura, marketing, soporte). Diferenciarse de la competencia y maximizar el crecimiento del usuario.

3. *Fase de Madurez*

- * El crecimiento se estabiliza.
- * El producto ha alcanzado la mayor parte del mercado objetivo, la competencia es alta y el margen de crecimiento disminuye.
- * El objetivo es optimizar procesos y reducir costos, manteniendo la lealtad del cliente y posiblemente adaptando el producto o creando derivados.

4. *Fase de Declive*

- * Las ventas y los beneficios empiezan a caer, ya sea por cambios en el mercado, tecnología obsoleta, competencia disruptiva, etc.
- * El objetivo es decidir si retirar el producto, actualizarlo o pivotear. Buscando reducir los costos al mínimo y extrayendo el máximo valor restante.
- * Termina en el momento en el que el producto se retira del mercado.

Planificación del producto ágil

En las metodologías ágiles, se tiene una visión del producto materializado en el Product Backlog, que contiene todas las características de la funcionalidad que requiere el producto que han sido identificadas hasta el momento. Esta visión se descompone en releases, las cuales son las entregas frecuentes que se hará al cliente. Cada release se construye a partir de sucesivas iteraciones.

Valor vs Desperdicio

La productividad de un Start-up no puede medirse en términos de cuánto se construye cada día, por el contrario, se debe medir en términos de construir lo correcto cada día y lo correcto es aquellos que agrega valor. Si no construimos valor, estamos invirtiendo esfuerzo en desperdicio.

Mínimo Producto Viable

Es un concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos. Se define como la **versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje valioso sobre los clientes con el menor esfuerzo**.

Ver lo que la gente realmente hace con el producto es mucho más confiable que preguntarle a la gente que harían.

Una premisa clave detrás de la idea del MVP es que usted produce un producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio.

Sus características:

1. Tiene el **valor suficiente para que las personas estén dispuestas a usarlo** o comprarlo.
2. Demuestra beneficios a futuro para retener a los usuarios.
3. Proporciona un ciclo de **retroalimentación** para guiar el proceso futuro.

Los errores comunes son:

1. Confundirlo con MMF o MMP. La MMF (Característica Comercializable Mínima) o el MMP (Producto Comercializable Mínimo) a diferencia del MVP se enfocan en **ganar**, en cambio un MVP se enfoca en el aprendizaje.
2. Enfatizar la parte de mínima y dejar de lado la parte viable. Si el producto no tiene la calidad suficiente, no será posible obtener la retroalimentación del cliente.
3. Hacer del MVP un producto final, es decir, entregar lo que consideran un MVP, y luego no hacer más cambios a ese producto, independientemente de los comentarios que reciban al respecto.

Un MVP permite validar diferentes hipótesis:

1. Hipótesis del valor: prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es realmente un producto correcto). Una métrica de prueba: tasa de retención.
2. Hipótesis de crecimiento: prueba cómo nuevos clientes descubrirán el Producto (ver si más personas desean usar el producto). Una métrica: tasa de referencia o Net Promoter Score (NPS).

Preparar un MVP:

1. Encontrar un nicho.
2. Crear un Roadmap realista. Definición de muy alto nivel de qué características del producto se podrían tener a cierta altura del año.
3. Investigar la competencia.
4. Pre-vender el MVP.
5. Testear las suposiciones.
6. Asegurarse que el MVP resuelve el problema correcto.
7. Focalizar en las funcionalidades principales.

Explicación MVPs, MVFs, MMFs con el dinosaurio Lean/Agile

1. UVP

- * Producto único variable.
- * Idea que necesita ser validada en el mercado.
- * Un producto nuevo tiene una única hipótesis de valor único: producto/servicio único.

2. MVP

- * Producto mínimo viable.
- * Se centra en su propuesta de valor única.
- * Es una hipótesis, podría ser lo suficientemente bueno para encontrar un mercado o no.
- * “Esto es genial, pero para poder usarlo necesito X”, X es diferente para cada cliente/usuario.
- * Si se repiten respuestas apuntando al mismo X entonces tiene sentido revisar la hipótesis de cliente, problema o solución.

3. MVP2

- * Se lo construye centrado en la nueva hipótesis basada en la nueva hipótesis basada en el aprendizaje reciente de Desarrollo de clientes generado por el anterior MVP.
- * Una vez que este sea exitoso, mediante feedback e investigación, encontramos áreas que pueden traer este crecimiento, donde definiremos MMF.

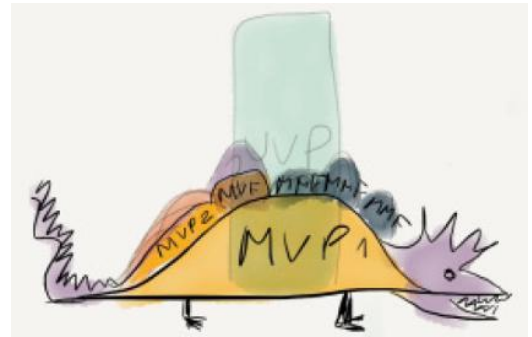
4. MMF

- * Características Mínimas Comercializables.
- * El objetivo es entregar valor, supone una alta certeza de que existe valor en esta área y que sabemos cual debe ser el producto para proporcionar valor.
- * Se divide una característica grande en MMF más pequeños por el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas.
- * La hipótesis se centra en una característica en lugar de un producto, tienes un área con alto potencial, pero con alta incertidumbre.

5. MVF

- * Característica Mínima Viable.
- * La característica mínima que aun puede ser viable para uso real y aprendizaje de los usuarios reales.
- * Si resulta exitosa, puede desarrollar más MMF en esa área para tomar ventaja,
- * Si no resulta exitosa, puede cambiar a otro enfoque hacia esa área de características, o buscar una ruta de crecimiento alternativa.
- * Es una versión mini del MVP.

El producto se cultiva en mercados inciertos al intentar varios MVPs. Cuando se logra ajustar el producto vamos experimentando con MMFs y MVFs en distintas áreas según el nivel de incertidumbre de negocio y de los requisitos de las áreas. Cada una de estas piezas que fuimos agregando y cortando son las US. Esas porciones más pequeñas pueden tener un valor comercial tangible o no.

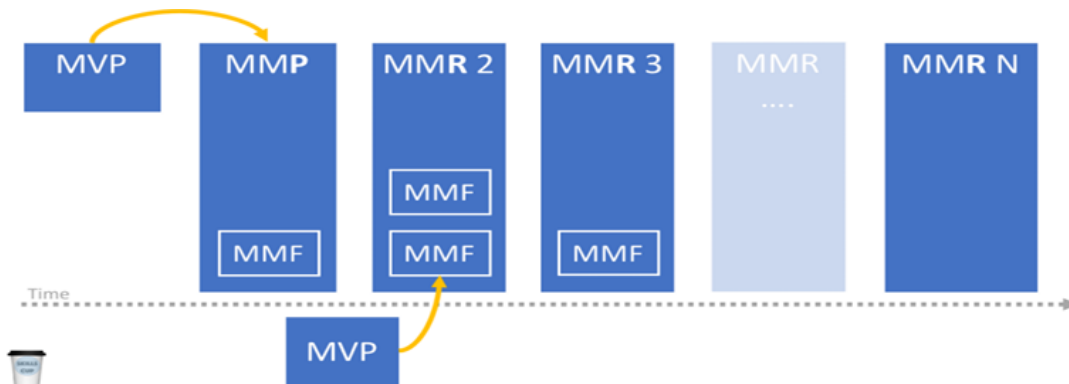


Resumen de los productos mínimos para la gestión de productos:

- MVP: versión de un nuevo producto creado con el menor esfuerzo posible.
- MVF: versión mini del MVP MRF. Características mínimas que debe tener un producto para salir al mercado.
- MMF: pieza más pequeña de funcionalidad que puede ser liberada. Es parte de MMR o MMP.
- MMP: primer release de un MMR dirigido a primeros usuarios. Mínimo producto comercializable.
- MMR: release de un producto con el conjunto de características más pequeño posible.

Las User Stories van a determinar el MVP que se desarrollará y que se debe validar.

Mientras menos tiempo se invierte, menos plata y menor el riesgo.



Relación entre MVP, MMF, MMP, MMR

MVP

- * Versión de un nuevo producto creado con el menor esfuerzo posible.
- * Dirigido a un subconjunto de clientes potenciales.
- * Utilizando para obtener aprendizaje validado.
- * Más cercano a los prototipos que a una versión real funcionando de un producto.
- * Concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos.
- * Producto real que puede ofrecer a los clientes y observar su comportamiento real con el producto o servicio.
- * “versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre clientes con el menor esfuerzo”.
- * Ver lo que la gente realmente hace con respecto a un producto que es mucho más confiable que preguntarle a la gente qué harían.

MMF

- * Es la pieza más pequeña de funcionalidad que puede ser liberada.
- * Tiene valor tanto para la organización como para los usuarios.
- * Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica.
- * Es parte de un MMR o MMP.

MMP

- * Primer release de un MMR dirigido a primeros usuarios (early adopters).
- * Focalizado en características clave que satisfarán a este grupo clave.

MMR

- * Release de un producto que tiene el conjunto de características más pequeño posible.
- * El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.
- * MMP = MMR1.

MVF – Característica Mínima Viable

- Versión mini del MVP.
- Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción de la característica.
- Debe proporcionar un valor claro a los usuarios y ser fácil de usar.
- Requiere recursos mínimos, los estándares de calidad de la industria y la producción deben guiar el diseño y la confiabilidad.
- Los resultados le ayudarán a tomar decisiones estratégicas sobre productos.

Valor vs Desperdicio

- Lean Thinking define la creación de valor como proveer beneficios a los clientes, cualquier otra cosa es desperdicio.
- Desperdicio → todo lo que no produce valor.
- Los requerimientos que no se utilizan son desperdicio.
- Hay que minimizar o eliminar los desperdicios.
- El éxito no es entregar un producto, el éxito se trata de entregar un producto que el cliente usará.
- La forma de hacerlo es alinear los esfuerzos continuamente hacia las necesidades reales de los clientes.

Método Lean Startup: busca reducir tiempo y costo de crear empresas usando la hipótesis y experimentación. Lo más importante del startup es el aprendizaje, debemos crear un bucle de feedback para alcanzar las expectativas del cliente. El éxito es entregar un producto que el cliente usará.

Dilema: La audacia de Cero

- A menudo es más sencillo recaudar dinero cuando se tiene cero ingresos, cero clientes y cero transacciones.
- Es determinante si se tiene algo o no, si no se tiene, simplemente se arriesga.
- Cero invita a la imaginación, incentiva a aplazar el lanzamiento de un producto hasta asegurar el éxito.
- Mientras más tenés para perder → menos vas a ganar.
- Use un MVP para experimentar con los primeros usuarios del mercado.
- Verifique su concepto probando todos sus elementos, comenzando por los más riesgosos.

Características de un MVP

1. Diseño: diseño adecuado con UX agradable y que satisface aspecto visual y de interacción.
2. Usabilidad: suficiente valor para que la gente lo use/compre. Útil para un público objetivo.
3. Confiabilidad: involucrar a los early adopters para que puedan confiar en la solución plenamente.
4. Funcionalidad: tiene las funciones necesarias para solucionar un problema específico. Satisface las demandas y permite evaluar las funciones a implementar más adelante

Es mejor un poco de todas que mucho de una.

	Urgente	No urgente
Importante	1.Hacer Incluir en el MVP	2. Planear Desarrollar para un lanzamiento beta
No Importante	3.Delegar Considerar integraciones de terceras partes.	4. Eliminar Remover del Roadmap del producto

MLP – Minimal Lovable Product

Es la versión más simple de un producto que genera una conexión emocional o deleite en los usuarios. Enfatiza la satisfacción del usuario y la diferenciación. Fomenta la lealtad y la adopción temprana.

Marca el nivel mínimo de diseño/desarrollo para que los clientes lo amen en lugar de simplemente tolerarlo. Crear un MVP que, además de proporcionar información al equipo de trabajo para el desarrollo futuro, apele a la emocionalidad del usuario para lograr lo más difícil: enamorarlo.

Claves para crear un MLP:

- a) **Reputación de marca.** Demostrar al usuario que nos importan las características que necesitan, que no se vea como un intento de lucro.
- b) **Productos destacados.** Destacar en el mercado para el financiamiento. Se necesita un producto atractivo.
- c) **Ciclo de retroalimentación del usuario.** Capitalizar el interés de los usuarios desde el principio a medida que se desarrolla. Se puede capitalizar sus intereses creando un flujo de retroalimentación.

Claves para definir un MLP

- 1) Entender y conocer al público objetivo.
- 2) Definir una propuesta que aporte valor a las necesidades y expectativas del usuario. Algo que nos diferencie.
- 3) Resolver al menos una de las necesidades del usuario de manera excelente.
- 4) Sorprender al usuario, añadiendo un factor sorpresa en la parte visual o funcional del producto.

Para poder crear un MLP debemos sentir curiosidad por lo que valora un cliente, conocer el mercado, esforzarse, etc.

FILOSOFÍA ÁGIL – MANIFIESTO ÁGIL

El Manifiesto Ágil es un documento redactado en 2011 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software. Consiste en una visión fundamentada por valores y principios ágiles.

Valores Ágiles

Individuos e interacciones por sobre Procesos y Herramientas

Es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos.

Los procesos ágiles intentan capitalizar las fortalezas de cada individuo, en lugar de intentar homogeneizar un equipo, para así lograr una armonía que permita fortalecer la unión y motivación dentro del equipo.

Los procesos y las herramientas deben ser un complemento para mejorar la eficiencia y éstos se deben adaptar al equipo y no al revés.

Software funcionando por sobre Documentación excesiva

Priorizar el software asegura que se tengan versiones estables, incrementables y mejoradas del software al finalizar cada iteración. Esto permite que a medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos, para que el equipo de desarrollo se asegure de estar trabajando siempre en aquellas funcionalidades de mayor valor para el cliente y aquellas que satisfagan sus expectativas. Solo se documenta aquello que agregue valor al producto y esté centrado en el cliente (valor agregado).

Colaborar con el cliente por sobre Negociación contractual

Esto permitirá no sólo tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente, sino también que el equipo ágil se integre a todas las partes interesadas en el software, para que en conjunto puedan trabajar y lograr los mismos objetivos. Reduce la dependencia de documentación extensiva ya que cada miembro del equipo participa en la toma de decisiones que solían requerir comunicación escrita.

Implica comprometer al cliente como parte importante del producto que se está construyendo.

No se trata de tener a un cliente lejano que no sabe lo que sucede durante el desarrollo, si no tener a uno que esté comprometido con cada entrega, sobre todo al momento de probarlo y dar el feedback.

A veces las negociaciones contractuales imponen ciertas restricciones que impactan de manera negativa en las partes, lo cual se traduce en un mayor distanciamiento en la relación equipo de desarrollo-cliente.

Responder al cambio por sobre Seguir un plan

Es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades, las cuales en un primer momento no se habían detectado.

Esto está ligado al entorno cambiante en el que se desempeñan los clientes que demandan un software, por lo que un equipo ágil está abierto a los cambios, para poder responder ante ellos de una manera rápida y eficiente entregando el mayor valor posible para el cliente y los usuarios, en lugar de apegarse y seguir un plan definido confeccionado en etapas tempranas de requerimientos.

Los principales valores de la gestión ágil son la inspección y la adaptación, diferentes a los de la gestión de proyectos tradicional: planificación y control que evite desviaciones del plan. (Los cambios son vistos de buena forma).

Analizando en concreto el proceso de desarrollo de un equipo ágil, al finalizar cada iteración, la experiencia de desarrollar el incremento genera cierto conocimiento en el equipo, que afecta a la planificación en los distintos niveles. De forma similar, mostrar o desplegar esos incrementos a los usuarios finales, también generará nuevo conocimiento para esos procesos de planificación.

Este nuevo conocimiento puede inducir a cambios, los cuales el equipo ágil deberá incluir en su planificación, de forma que siempre se apunte a entregar un producto con el mayor valor posible.

Principios del Manifiesto Ágil

1. Nuestra mayor prioridad es satisfacer al cliente.

Ligado con el valor 3 y 4, debido a que un equipo ágil desarrolla y entrega funcionalidades en el orden especificado por el Product Owner. Es el rol que determina la prioridad e importancia de cada funcionalidad o feature, de manera que en cada incremento o Release del producto, se esté retornando al cliente el mayor valor posible.

2. Aceptar que los requisitos cambien.

Ligado a los valores 3 y 4, donde se plantea que el equipo ágil está abierto al cambio, al no seguir un plan estructurado que se confecciona en etapas tempranas, como plantea la metodología tradicional. Permite que el equipo sea flexible, y responda siempre ante las necesidades cambiantes de los clientes y/o usuarios.

3. Entregar software funcional frecuentemente.

Se relaciona con el valor 2, donde es imprescindible que un equipo ágil al finalizar cada iteración (las cuales duran periodos cortos de entre 1 a 4 semanas), haya sido capaz de transformar uno o más requerimientos en código desarrollado completamente, testeado y potencialmente desplegable. Debe cumplir con la DoD.

4. Técnicos y no técnicos trabajando juntos durante todo el proyecto.

Ligado con los valores 1 y 3, y lo que plantea es la gran importancia de la participación del rol conocido como Product Owner, en conjunto con el equipo de desarrollo de software. Este rol, debe ser capaz de tomar decisiones sobre el producto, y que además sea capaz de representar el interés y necesidades comunes de todos los usuarios.

5. Desarrollamos proyectos en torno a individuos motivados.

Ligado al valor 1, donde la filosofía asume que aquellos individuos que participan en un proyecto de desarrollo del software deben estar lo suficiente motivados y comprometidos con el objetivo de este.

6. El método más eficiente de comunicar información es conversación cara a cara.

Relacionado con el valor 1, este principio tiene un sustento psicológico, el cual expresa que las palabras expresadas como sonido o texto, sólo representan un bajo porcentaje de la comunicación que se desea lograr hacia otra persona, siendo que la mayor parte está ligada a gestos, expresiones faciales, movimientos corporales, retroalimentación rápida, etc.

7. El software funcionando es la principal medida de éxito.

Ligado con el valor 2, donde se plantea que la mejor forma de demostrar el avance en un proyecto es a través de la entrega de software funcional y de valor para el cliente.

8. Los procesos ágiles promueven el desarrollo sostenible.

Ligado con los valores 2 y 4, esto apunta a la estabilidad en equipos de trabajo a lo largo del tiempo, lo cual se traduce en una mejora continua de relaciones y formas de trabajo del equipo. Se propone que los equipos se mantengan a lo largo del tiempo para mejorar su dinámica de trabajo.

9. La atención continua a la excelente técnica y al buen diseño mejor la Agilidad.

Se relaciona con el valor 2, y manifiesta que la calidad del producto no es un aspecto negociable. Es importante que a medida que el software se desarrolla, se atiendan las necesidades primordiales del cliente y se responda a ellas, entregando al cliente código de calidad.

10. La simplicidad es esencial.

Ligado con los valores 2 y 3, apunta que, como equipo de desarrollo, nos centremos en aquellas necesidades que realmente aporten un valor al cliente, y no en perder el tiempo en detalles o aspectos que no se traducen en satisfacer requerimientos de los usuarios, ni hacer desarrollos de más que no estén especificados como necesidad del cliente.

11. Las mejores arquitecturas, requisitos y diseños emergen de equipos auto-organizados.

Ligado al valor 1, expresa que los equipos de desarrollo ágiles a diferencia de organizaciones jerárquicas tradicionales, donde la dirección/gerencia o los jefes son quienes imparten órdenes, y todos deben acatarlas, bajo esta filosofía los integrantes del equipo deben ser proactivos, aportando soluciones, conceptos, herramientas, etc. útiles para el equipo.

12. A intervalos regulares el equipo reflexiona y de acuerdo a esto ajustan su comportamiento.

Relacionado a los valores 1 y 4, este principio se basa en los dos pilares de los procesos empíricos (inspección y adaptación), los cuales permiten a los equipos que cada determinado periodo de tiempo evalúe distintos aspectos concernientes a su desempeño.

Un equipo ágil debe inspeccionar su forma de desempeño para adaptar aquellos aspectos que sean necesarios corregir, mejorar o incorporar al equipo, de manera que el comportamiento del equipo como un todo mejore, y de esa forma se pueda responder a las necesidades cambiantes de los clientes.

Filosofía ágil

Ágil es una ideología con un conjunto definido de principios que guían el desarrollo de un proyecto. No es una metodología ni proceso.

El objetivo primordial es entregar software de forma frecuente en un entorno cambiante. Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación, ya que estos encuentran imposible predecir cómo un sistema afectará sus prácticas operacionales o qué cambios habrá en el entorno que puedan dejar el sistema completamente obsoleto.

El desarrollo ágil adopta el ciclo de vida iterativo-incremental, donde el concepto de empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad.

Usualmente los incrementos se crean cada 2 o 3 semanas y se ponen a disposición del cliente. El cliente se encuentra inmerso en el proceso, ofreciendo una rápida retroalimentación que permiten una inspección y adaptación sobre los requerimientos cambiantes. Por otro lado, se reduce la cantidad de documentación con el uso de comunicaciones frecuentes e informales con el cliente en lugar de reuniones formales con documentos escritos.

Los métodos ágiles son adaptables en lugar de predictivos y están orientados a la gente en lugar del proceso.

Nace buscando un equilibrio entre hacer Software siguiendo rigurosamente un proceso donde a la larga termina siendo más importante cumplir con el proceso que entregar software de calidad y, yéndonos al otro extremo, hacer software a la ligera.

La filosofía ágil se materializa en el manifiesto ágil. Éste está basado en que la forma de trabajo va a ser sustentada en los conceptos de los procesos empíricos, bajo 3 pilares fundamentales, la transparencia, la inspección y la adaptación.

Pilares del empirismo

Transparencia: permite crecer como equipos y transformar el conocimiento que cada miembro del equipo tiene implícito en conocimiento explícito, es decir, que pase a formar parte del conocimiento de todo el equipo. Esto ayuda a que los procesos empíricos puedan fluir y generar esa experiencia necesaria para que el equipo evolucione. La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

Inspección: puede realizarse para el producto, procesos, aspectos de personas, prácticas y mejoras continuas. Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados. La inspección permite la adaptación.

Adaptación: se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. Nos permite hacer ajustes en aspectos del proceso que se desvíen fuera de los límites o si el producto es inaceptable. Los ajustes deben realizarse lo antes posibles para minimizar la desviación normal. El equipo debe adaptarse en el momento que aprenda algo nuevo por medio de la inspección.

¿Cuándo es aplicable Agile?

Ágil no es una metodología o un proceso, ni tampoco un framework completamente definido, sino que es una ideología con un objetivo definido de valores y principios que guían el desarrollo del producto. Definen prácticas de cómo hay que organizarse, es decir, cómo debe organizarse el grupo.

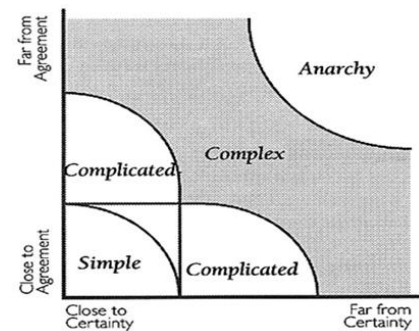
Los frameworks ágiles son distintos marcos de trabajo para implementar los principios y valores ágiles y generalmente son orientados a la gestión de proyectos y no al desarrollo.

Las metodologías ágiles son aplicables en el contexto de lo complejo (dan buenos resultados), en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial.

La investigación actúa en el campo de la “anarquía”, donde existe una gran incertidumbre.

Usualmente y no es una regla general, el mantenimiento se encuentra en el campo de lo relativamente simple.

Ser ágil no es ser indisciplinado, no hay que recaer en lo que la imagen representa.



¿Por qué ir a ágil?

- Acelera entregas de producto con valor para el cliente.
- Permite adaptarse fácilmente a cambios primordiales y requerimientos.
- Permite alinear los objetivos organizacionales y de TI.
- Reduce riesgos de proyecto.
- Mejora las relaciones en el equipo.
- Reduce costos de proyecto.

El triángulo ágil

El triángulo ágil apunta a lo que realmente tiene valor que es el software funcionando para el cliente, por lo que se centra más en la calidad del producto, es decir, en la calidad intrínseca (las características de calidad que nosotros le ponemos al producto para que el producto pueda satisfacer al cliente) y la calidad extrínseca (el valor que el producto tiene para el cliente).

El triángulo ágil (modificado) apunta a un producto de software que pone a las personas (que tienen ciertas características como confianza, motivación, etc.) en el lugar más importante ya que las personas son las que hacen el software. La intención de mejora continua que tienen las personas en el proyecto del producto y del proceso.



- Las personas son lo más importante, donde se prioriza que haya comunicación, motivación, confianza, excelencia y autogestión.
- Debajo encontramos el pie, en el cual tenemos la mejora continua. Es el triángulo de resultados, teniendo como pie la calidad y al valor.
- Del otro lado de la base podemos encontrar el alcance, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.

REQUERIMIENTOS ÁGILES

Lo más difícil del software es decidir qué software queremos construir, ya que las cuestiones tecnológicas no son el impedimento, sino que los proyectos de software fracasan debido a que no se cumplen con las expectativas del cliente; lo que se construye no es lo que el usuario quería o necesitaba. Este problema se advierte en la crisis del software.

Probablemente una de estas razones es no asimilar la complejidad que supone hacer software y que la etapa de relevar requerimientos suele ser subestimada.

También, es muy común que haya problemas de comunicación entre los clientes y el equipo. Cuando ninguno de los dos domina la comunicación, el dominio fracasa. Lo óptimo es que haya una comunicación de ambas partes, ya que los proyectos suelen fracasar si solo una parte domina la comunicación y asignación de recursos.

Como usuario, ver “early versions” o entregas tempranas es una ventaja ya que permite que nos surjan nuevas ideas o que nos demos cuenta de que no es exactamente lo que queríamos antes de que el proyecto siga avanzando por un camino equivocado.

Gestión Ágil de Requerimientos de Software

El fondo de la cuestión está en los principios y valores explicados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos en etapas finales. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido.

Pilares de los Requerimientos Agile:

- * **Usa el valor para construir el producto correcto:** Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio. Hay que construir un software que le haga la vida más fácil a las personas que lo utilizan.
- * **Determinar que es “solo lo suficiente”:** Los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Se tiene que empezar con lo mínimo necesario para generar retroalimentación y alimentar el empirismo, y de allí, se avanza continuamente.
- * **Usar historias y modelos para mostrar qué construir:** la parte del desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio agile de “técnicos y no técnicos trabajando juntos”. Se busca trabajar con un referente que esté del lado del negocio.

La etapa de capturar los requerimientos se transforma en una etapa de trabajo muy integrado entre el cliente y nosotros. Por eso el mejor medio de comunicación es la comunicación cara a cara porque la idea es armar equipos de trabajo en los que con técnicas de descubrimiento (User Stories) se puede lograr crear la visión de qué producto se quiere construir y a partir de ahí, avanzar.

El enfoque de requerimientos ágiles debe estar adherido al manifiesto ágil, es decir, a cumplir con los valores y con los principios que el manifiesto plantea.

La gestión de requerimientos está vinculada con mantener los requerimientos íntegros, bien identificados, bien trazados y que si cambian los requerimientos se actualicen todos los lugares donde se tengan que actualizar esa definición. El desarrollo de requerimientos es la parte que tiene que ver con identificar, especificar y validar los requerimientos.

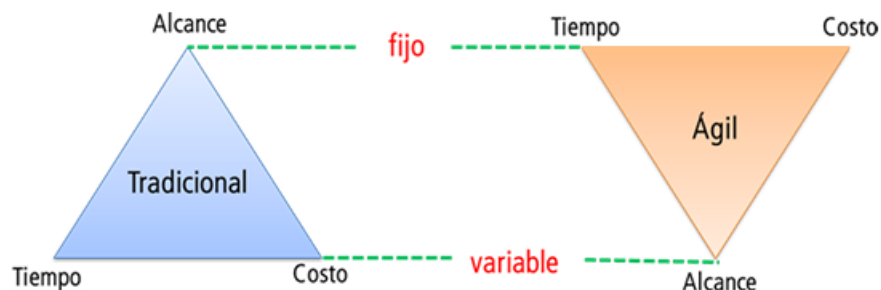
Para no agregar funcionalidad sin sentido, la gestión ágil agrega el concepto de Product Owner, quién es el que mejor conoce sus necesidades, determina el orden de los requerimientos que el considere más importantes en el Product Backlog.

El Product Owner debe estar disponible para funcionar. Va en línea con el principio de “Just in Time”. Esta agilidad en cuanto a la comunicación hace no necesario la documentación excesiva.

El principio de comunicación cara-a-cara se justifica en: permitir que fluya la información vocal, subvocal, gestual con realimentación rápida.

La triple restricción - enfoque Ágil

En el enfoque ágil, se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este. Entonces, el enfoque trata de dejar el tiempo y los recursos fijos, teniendo en cuenta “cuánto software” le puedo entregar en tanto tiempo y con tal equipo de desarrollo.



Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia, los recursos y el tiempo.

BRUF – Big Requirements Up Front

A lo que apunta BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.

Es prácticamente imposible conocer al 100% todas las necesidades de los usuarios. Además, esas necesidades que tienen los usuarios al momento de ser reveladas, con el tiempo probablemente irán cambiando, con lo cual, la priorización es un factor fundamental.



Product Backlog

La gestión ágil busca contrastar la situación planteada con anterioridad donde gran parte del software no se termina utilizando. El Product Backlog es una lista priorizada de características y requerimientos del software.

Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.

En contraposición con el enfoque tradicional, donde teníamos un cliente que no estaba dispuesto a comprometerse y dejaba la priorización de los requerimientos al equipo que hace el software, se terminaba construyendo un producto que no dejaba satisfecho al cliente. En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado.

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



Con esta situación se permite la compensación de no tener documentos de requerimientos formales escritos, detallando todos los puntos. Ágil ofrece una compensación al afirmar que la mejor comunicación es el cara a cara y un cliente participe en el equipo.

Requerimientos Just in Time

La gestión ágil de requerimientos sostiene los **requerimientos justo a tiempo**, asociados a los principios de eliminar desperdicio y diferir compromisos, dentro de la filosofía Lean.

Se empieza con una visión de producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. **El product backlog nunca está al 100%.**

En el enfoque tradicional se aplica el “Up Front Specification”, mediante el cual se especifica todos los requerimientos antes de comenzar el desarrollo. Esto implica que un error en la captura de requerimientos tiene un alto impacto. En cambio, los requerimientos en ambientes ágiles al ser consensuado continuamente con el cliente, el impacto que se tiene es menor.

El producto no va a estar especificado al 100% desde el principio ni muchísimo menos, sino que se van a ir encontrando requerimientos y describiendo los requerimientos que hacen falta conforme haga falta. “ni antes, ni después”.

Fuente de requerimientos

Los requerimientos ágiles, basados en el principio de que el mejor medio de comunicación es el cara a cara, promueven que los requerimientos de software se obtienen conversando cara a cara con el cliente.

En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo (sería lo que en el gráfico dice “papel”).

Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas. Esto permite reducir la cantidad de documentación formal. La elicitación de requerimientos es un proceso que requiere de conversar de forma personal, analizando lo que los clientes dicen y los gestos que manifiestan.

El desarrollo ágil no implica la no documentación o la no especificación de requerimientos, sino que prioriza documentar y especificar aquellos que sean de valor para el cliente, antes que hacer toda una ERS errónea.

Momento de la captura de requerimientos

En el enfoque tradicional, los requerimientos son definidos al principio del proyecto. El flujo de trabajo de requerimientos tiene un gran peso en la fase inicial y de elaboración.

En el enfoque ágil, los requerimientos son relevados continuamente durante todo el proyecto. El inicio de la construcción del software se da en un contexto en el cual los requerimientos no están completamente definidos, pero se tiene una visión.

Contenedor y persistencia de los requerimientos

En el enfoque tradicional, la ERS contiene los requerimientos a lo largo del ciclo de vida del producto todo el tiempo. En caso de que surjan cambios, es necesario actualizar la ERS. Es el documento de más alto nivel que define las características del sistema.

En el enfoque ágil, se utiliza al Product Backlog como un contenedor transitorio de los requerimientos, ya que una vez implementados estos se “persisten” en el producto funcionando. El detalle de valor queda persistente en los casos de prueba que se construyen en el Testing.

Tipos de requerimientos

La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

A nivel de la gestión ágil, se trabaja con requerimientos de negocio y requerimientos de usuario.

Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.

Al usuario le interesa el requerimiento de negocio. El valor de negocio es el requerimiento de negocio, pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio.

¿Qué debemos asumir?

- Los cambios son la única constante. Todo el tiempo va a haber cambios en los requerimientos.
- Stakeholders: no son todos los que están. Debemos saber quiénes son los involucrados y los involucrados son todos lo que tienen algo que decir respecto al producto. El PO es el representante de todas las personas y el que se comunica con todas ellas para luego ser él mismo el que se comunica con el equipo que hace el software.
- Siempre se cumple eso de que: “El usuario dice lo que quiere cuando recibe lo que pidió”.
- No hay técnicas ni herramientas que sirvan para todos los casos. Debemos detectar para cada situación cuál es la herramienta más adecuada.
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de “valor”.

Principios ágiles relacionados con Requerimientos ágiles

1. La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes.
2. Recibir cambios de requerimientos, aún en etapas finales.
4. Técnicos y no técnicos trabajando juntos todo el proyecto.
6. El medio de comunicación por excelencia es cara a cara.
11. Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados.

USER STORIES

Las user stories son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.

Funciona como un recordatorio para el equipo de desarrollo y el PO de qué tiene que conversar acerca de la necesidad en términos de negocio para la construcción de una característica del producto que aporte valor. Es un mecanismo para diferir una conversación.

Es un requerimiento a nivel de usuario, un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario. Las US son escritas por el PO.

Internamente, las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.

Se dice que son multipropósito:

- Representan una necesidad del usuario, de forma no detallada.
- Conforman una descripción del producto.
- Son utilizadas como ítems de planificación.
- Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
- Junto con el código ejecutable, permiten documentar el producto.

La problemática que motivó en la definición de esta técnica es la dificultad de comunicar lo que hará el sistema a todas las personas afectadas. Cada una tiene un interés respecto a esta información.

Partes de una User Story

1. Tarjeta

Es una descripción de la historia, utilizada para planificar y como recordatorio. Tiene una sintaxis particular que es la siguiente:

<< Frase Verbal >>
Como << nombre del rol >> yo deseo << actividad >> para << valor de negocio >>

Nombre del rol: representa quién está realizando la acción o quién recibe el valor de la actividad.

Actividad: representa la acción que realizará el sistema.

Valor de negocio: representa el por qué es necesaria la actividad. Justifica la razón de que el PO escriba la US.

Frase Verbal: forma corta de referencia la US. Representa la funcionalidad que se expresa en la US.

Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Se deben redactar independientes a la implementación.

Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa.

2. Conversación

Es la parte más importante de la US, ya que explicita la comunicación entre el PO y el equipo de desarrollo que permite compensar la falta de especificación y detalle. Son discusiones acerca de la historia que sirven para desarrollar los detalles de la US.

3. Confirmación

Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conformar a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Ventajas

- Prioriza la comunicación verbal por sobre la escrita (la conversación es la parte más importante).
- Son comprensibles tanto para el cliente/usuario (Product Owner) como para el equipo de desarrollo.
- Tienen el tamaño justo para permitir planificar.
- Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios.

Desventajas

- En proyectos grandes, es difícil entender la relación entre US, al ser independientes entre sí.
- No son apropiadas para equipos demasiado grandes, por el tema de priorizar la conversación verbal.

Product Backlog y User Stories

El Product Owner prioriza las historias en el Product Backlog.

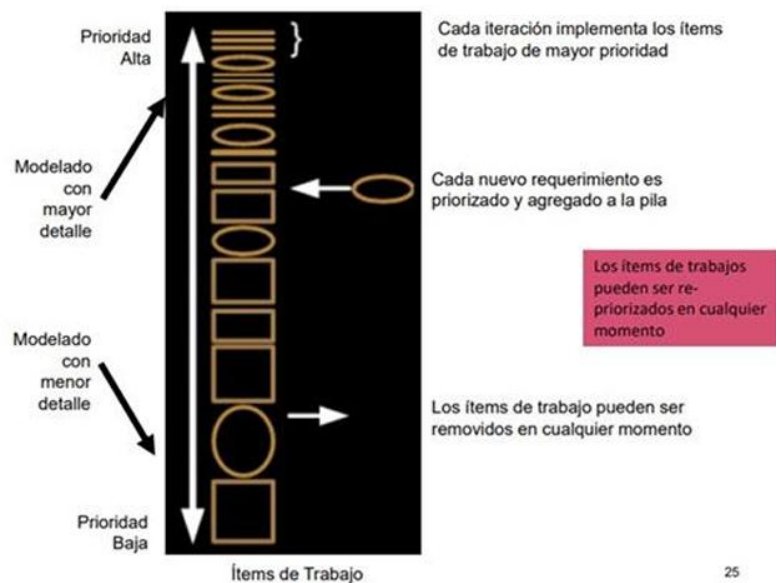
El Product Backlog es una pila de PBIs (Product Backlog Items) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (U.S. con granularidad fina), y las de menor prioridad (U.S. de granularidad gruesa, sobre las cuales no se tiene mucho detalle ya que no se consideran indispensables) al fondo.

En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner, quien es el encargado de realizar el proceso de priorización.

En cualquier momento del proyecto se van agregando nuevas U.S. y actividades a realizar, según se vayan detectando nuevos requerimientos.

Para quitar un U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo (Story Point / iteración), cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

Nunca encontraremos el 100% de los requerimientos de usuario en el Product Backlog.



25

Porciones Verticales

Las U.S. son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos.

Debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.

Relación con la arquitectura en capas

La User es una porción vertical, la cual abarca todas las capas de arquitectura: capa de datos, lógica de negocio y capa de presentación. Esto permite entregar una característica terminada al cliente.

Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener pocas historias, pero muy grandes.

El tamaño de la User Story se mide en Story Points.

Hay 3 aspectos que determinan el tamaño de una US:

- **Complejidad:** hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en US más pequeñas. Como algoritmos complejos, cálculos, gran cantidad de campos, muchas validaciones, demasiados filtros.
- **Esfuerzo:** es el tiempo en horas de trabajo que requerirá la implementación.
- **Incertidumbre:**
 - * Técnica: cuando se tiene dudas en el dominio de la solución, como el uso de una tecnología.
 - * De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Story Point

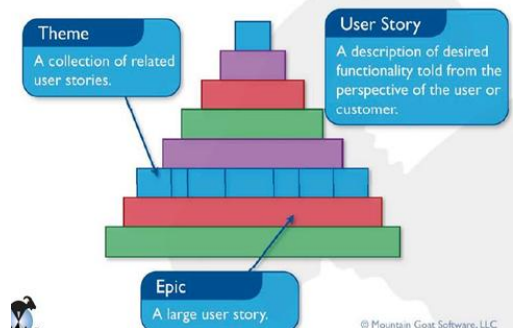
Son unidades de medida relativas (no importa el valor en sí, sino su comparación), las cuales miden el tamaño de un producto, NO de una medida basada en el tiempo (por ejemplo, horas hombre). Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores.

Lo relativo surge a partir de que se comprobó que las personas no saben estimar en términos absolutos, sino que les resulta útil la comparación de tamaños.

Niveles de Abstracción

1. **User Story:** cumple con el criterio de Ready (DoR) para ingresar a una iteración.
2. **Épica:** es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.
3. **Temas:** son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Stories, themes and epics



Modelado de Roles

Se utilizan las tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario.

El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.

Técnicas adicionales:

- * **Personas:** específica acerca de la descripción de la persona concreta que va a utilizar el sistema.
- * **Personajes extremos:** qué personas en términos extremos podrán utilizar el software.
- * **Proxies (Usuarios Representantes):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Criterios de Aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que esté debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Debe estar escrita en términos objetivos y concretos.

El criterio de aceptación es información concreta que tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no. Debemos saber si el PO nos va a aceptar esta característica implementada o no, porque la va a aceptar si respeta los criterios de aceptación.

Deben definir una intención, no una solución. Son independientes de la implementación. Relativamente de alto nivel, no es necesario que se escriba cada detalle.

Pruebas de Aceptación

Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación.

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

DoR – Definition of Ready

Es una medida de calidad que determina si la US está en condiciones de entrar a una iteración de desarrollo.

Para que la US sea implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

***INVEST Model**

Es un modelo de calidad que nos ayuda a verificar si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

- **Independent:** la US es calendarizable e implementable en cualquier orden. No existe una dependencia con otra US para desarrollarla o para mostrarla al PO.
- **Negotiable:** las US no son contratos escritos, son pequeñas descripciones de funcionalidades donde se describen las necesidades de los usuarios, y los detalles se dejan para negociar en conversaciones futuras.
- **Valuable:** una US debe ser valorable para los clientes o usuario.
- **Estimatable:** la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo. Sin poder estimarla no es posible determinar si se puede finalizar en una iteración por lo tanto no puede ser construida.
- **Small:** la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración
- **Testable:** la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada.

DoD – Definition of Done

Determina si la US está terminada. Son los criterios que establecen cuándo una US ha sido implementada, de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. En el momento en que una US cumple con todo lo pactado en esta DoD, se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Permite crear transparencia, proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.

Consta de in checklist de condiciones que deben cumplirse. Puede ser definido por la organización o por el equipo.

Spikes

Son un tipo especial de US que se **producen por la incertidumbre** que la misma presenta, la cual imposibilita que pueda ser estimada y, por lo tanto, no cumple con la definición de listo.

Una vez resuelta la incertidumbre, la Spike se convierte en una o más US.

Es una característica más del producto a la cual el equipo le debe dedicar tiempo para:

- Familiarizarse con una nueva tecnología o dominio.
- Investigar y prototipar para ganar confianza frente a:
 - * Riesgos tecnológicos.
 - * Riesgos funcionales, no está claro cómo debe reaccionar el sistema para satisfacer las necesidades del usuario.

Spikes Técnicas

Utilizadas para evaluar diferentes enfoques técnicos y tecnológicos en el dominio de la solución. Está asociado a la implementación de la funcionalidad. Nueva tecnología.

Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo. Dependen de los técnicos (nosotros).

Spikes Funcionales

Utilizada cuando hay cierta incertidumbre respecto de cómo el usuario interactúa con el sistema, usualmente son mejor evaluadas con prototipos para obtener retroalimentación de los usuarios involucrados.

Dependen de definiciones del negocio, del PO.

Lineamientos para Spikes

- Estimables, demostrables, y aceptables.
- La excepción, no la regla.
 - * Toda historia tiene incertidumbre y riesgos.
 - * El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
 - * Los spikes deben dejarse para incógnitas más críticas y grandes.
 - * Utilizar spikes como última opción.
- * Implementar la spike en una iteración separada de las historias resultantes.
- * Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

ESTIMACIONES ÁGILES

Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto. Se relaciona con los objetivos de negocio, compromiso y control.

Según McConell una estimación en el contexto de software es una predicción de cuánto tiempo durará o costará un proyecto. Considerada una de las actividades más complejas en el desarrollo de software.

Las estimaciones no son planes ni compromisos, sino que sirven de base para planificar, pero el plan final no tiene por qué coincidir con lo estimado.

En los equipos ágiles, features/stories son estimadas usando una medida de tamaño relativo conocida como Story Points. Existen una serie de consideraciones:

- * Son medidas relativas no absolutas.
- * No es una medida basada en el tiempo.

La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones). Al ser de esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más preciso posible (a diferencia de las metodologías tradicionales, donde se estima individualmente y se depende sólo de ese miembro).

Existe una serie de consideraciones importantes:

- * Son medidas relativas no absolutas.
- * No es una medida basada en el tiempo.
- * Las personas no saben estimar en términos absolutos, somos buenos comparando.
- * Comparar es generalmente más rápido.
- * Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- * Se emplea mejor el tiempo de análisis de las Stories.

Para las estimaciones se deben tener en cuenta tres elementos a la hora de estimar: complejidad, esfuerzo e incertidumbre.

Estas tres variables son las que conforman al Story Point y de esta manera podemos comparar las US.

Tips respecto de las estimaciones

- * Si las estimaciones se utilizan como compromisos son muy peligrosas y perjudiciales para cualquier organización.
- * Lo más beneficioso en las estimaciones es el “proceso de hacerlas”.
- * La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no.
- * La estimación puede servir como una gran protección para el equipo.

¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de la US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes la serie de Fibonacci.

Al ser estimaciones de tamaños relativos, (es decir, se estima realizando comparaciones de los tamaños de las US) lo importante en este proceso no es el valor que se asigna a cada US en particular, sino más bien que las estimaciones sean consistentes en el conjunto. Es decir, para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, pero lo que importa es que, en un mismo equipo, una US de 5 puntos, siempre sea más chica que una de 8.

Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija (concepto de Timebox). Por esto, es que no se recomienda estimar el tamaño de las US en horas, para no confundir con el esfuerzo.

Tamaño vs Esfuerzo

La palabra tamaño refiere a cuan grande o pequeño es algo. Es una medida de la cantidad de trabajo necesaria para producir una feature/story.

El tamaño indica:

- ✓ Cuán compleja es una feature/story.
- ✓ Cuánto trabajo es requerido para hacer o completar una feature/story.
- ✓ Y cuán grande es una feature/story.

Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación, conlleva horas lineales (horas únicamente de trabajo, sin tener en cuenta almuerzo, baño, leer emails, etc).

No se debe confundir esfuerzo con calendario, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

Velocity

Velocidad o Velocity es una medida (métrica) del progreso de un equipo. Se calcula (no se estima) sumando el número de story point asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo (DoD), y además el Product Owner la ha aceptado.

Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración (no existe el concepto de casi completo, es una gestión binaria, es decir, se completó o no se completó la US). Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará, debido a que en unos pocos días probablemente se complete la US pendiente, y se pueda continuar con otras US planificadas en la iteración.

Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones. Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo.

Esta métrica permite retroalimentar el proceso de estimación, ya que un equipo maduro y con mucho trayecto, puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

Reestimación

Estimar nuevamente una US es aconsejable sólo cuando la opinión acerca del tamaño relativo de una US ha cambiado, debido a que en el proceso de estimación se han detectado otras U.S. con tamaños más grandes o pequeños. (esto tiene que ver con mantener la consistencia de tamaños a la hora de estimar). No se debería reestimar porque no se detecta un progreso como el esperado (velocidad del equipo), sino que se debe dejar que la velocidad a lo largo de sucesivas iteraciones equilibre esas inexactitudes.

Planificación

En ambientes ágiles, es necesario que quien planifica el proyecto de desarrollo de software, sea capaz de analizar el contexto del proyecto cada cierto periodo y hacer aquellos ajustes que sean necesarios, para encauzar los esfuerzos de manera correcta, y poder lograr los objetivos planeados.

“Planning is everything. Plans are nothing” → esto apunta a que lo importante de planificar, es el proceso en sí. No el hecho de realizar una documentación exhaustiva que respete determinados templates. Sí debe quedar registrado de alguna manera, idealmente bajo templates, pero sólo para servir de ayuda de memoria. Lo importante es el proceso.

Los equipos ágiles afrontan esta situación, planifican al menos en 3 niveles:

Planificación del Release

Se da al inicio del proyecto, donde se analiza y determina qué US, épicas o temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre, de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.

Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto (la cantidad de sprints necesarios para obtener el release).

Planificación de Iteración

Se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada. Esto se hace teniendo en cuenta el tiempo y la velocidad del equipo.

Durante esta planificación, se discute sobre las tareas que serán necesarias para transformar una US en código funcional y testeado.

Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Tamaño

Dado que el tamaño es lo que se compara, lo definimos como la medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para hacer o completar una Feature/US. También el tamaño es función de la incertidumbre de dicha US, definida la incertidumbre como la falta de información.

El tamaño no es lo mismo que el esfuerzo (el “quien” define esta diferencia), ya que este último indica las horas humano lineal requeridas para el desarrollo de la US. Por lo tanto, una US de mismo tamaño puede implicar esfuerzos diferentes en función de la persona que se esté evaluando, debido a que el esfuerzo varía dependiendo de las habilidades, conocimientos, experiencia, familiaridad con el dominio, etc.

Story Point

Es una unidad de medida específica (del equipo), de complejidad, riesgo y esfuerzo, es lo que “el kilo” a la unidad de nuestro sistema de medición de peso. Da idea del “peso” de cada story y decide cuán grande (compleja) es.

Es una unidad de estimación que especifica la complejidad, incertidumbre y esfuerzo propio del equipo respecto a una US en términos relativos respecto a otra US.

Complejidad: que tan compleja es implementar esa US. Relacionada a la cantidad de partes y relaciones entre estas.

Esfuerzo: es importante que estime la persona que va a hacer el trabajo, ya que el esfuerzo es distinto entre dos personas. Son las horas lineales que se requieren para estimar US.

Incertidumbre: es cuánta información falta para ser más adecuados en la estimación. Existe una incertidumbre técnica que es más sobre si se sabe cómo llevar a cabo lo explicitado con respecto a la tecnología a implementar y ese tipo de cosas. Y tenemos incertidumbre del negocio, si entendemos que es lo que el cliente pide y cómo llevarlo a cabo.

Poker Planning o Poker Estimation

Popular entre los Agile practitioners, publicado por Mike Cohn.

Resulta de la conjunción de diferentes métodos como el de juicio experto, analogía y desagregación, basado en que 4 ojos ven más que 2. En contraposición al método de juicio experto tradicional, asume que las personas que desarrollan el producto deben ser aquellas que estimen.

El equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. El Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

Escala

- * **Valor 1 o ½:** son los valores más bajos y se aplican a una funcionalidad pequeña.
- * **Valor 2-3:** funcionalidad pequeña a mediana.
- * **Valor 5:** funcionalidad media.
- * **Valor 8:** funcionalidad grande, donde nos preguntamos si es posible dividir.
- * **Valor 13 o más:** necesariamente hay que dividir ya que no ingresa en el sprint.

Prerrequisitos

- * Lista de features/stories a ser estimadas.
- * Cada estimador tiene un mazo de cartas.

Pasos

1. Determinar la **base story** (canónica) que será usada para comparar con las demás stories.
 - 1.1 La story a ser estimada se lee a todo el equipo.
 - 1.2 Los estimadores discuten la story, haciendo preguntas al PO.
 - 1.3 Cada estimador selecciona una carta y pone la carta abajo en la mesa.
 - 1.4 Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
 - 1.5 Si todos eligen el mismo valor, ese es el estimado. Si no, los estimadores discuten sus resultados.
2. Se toma la próxima story, se discute con el Product Owner.
3. Cada estimador asigna a la story un valor por comparación contra la base story. "Cuán pequeña/grande, compleja, riesgosa es esta story comparada con Story Z?".

Consideraciones

- * Para realizar una estimación se debe tener en cuenta el criterio de Done y no sólo la tarea de programación.
- * El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es timeboxing.
- * La US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Ventajas

- * Permite opiniones multidisciplinarias de todo el equipo, lo cual enriquece la estimación.
- * Al generar debates y discusiones, permite lograr una mejor comprensión del dominio en los miembros del equipo.
- * Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que estimar de forma grupal.
- * Es una actividad que resulta divertida para el equipo.

Diferencias con las estimaciones en ambientes tradicionales

	Tradicional	Ágil
Dimensiones	Alcance fijo, se calcula tiempo y recursos	Tiempo y recursos fijos, varía el alcance
Precisión	Busca alta presión, es la base de la planificación	Se aceptan estimaciones flexibles, no excesivamente precisas
Tipo de estimación	Estimación absoluta	Estimación relativa
Quién estima	Líder o experto	Equipo
Relación con planificación	Estimación como promesa	Estimación como probabilidad
Momento de estimar	Al inicio del proyecto	Al inicio de cada Sprint

FRAMEWORK SCRUM

SCRUM es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

Scrum se basa en el empirismo y el pensamiento Lean. El empirismo afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa. El pensamiento Lean reduce los desperdicios y se centra en lo esencial.

Scrum emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo.

Scrum requiere un Scrum Master para fomentar un entorno donde:

1. Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.
2. El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
3. El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
4. Repetir.

Son equipos autogestionados, es decir que no hay alguien que se encargue de la coordinación de este.

Scrum involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.

Scrum combina cuatro eventos formales para la inspección y adaptación dentro de un evento contenedor, el Sprint.

Pilares de SCRUM

Transparencia: el proceso y el trabajo emergente debe ser visible para aquellos que realizan el trabajo, así como para quienes reciben el trabajo. Con scrum las decisiones importantes se basan en el estado percibido de sus tres artefactos formales. La transparencia permite la inspección. La inspección sin transparencia genera engaños y desperdicios.

Inspección: los artefactos de scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas o problemas potencialmente indeseables. Para ayudar con la inspección, Scrum proporciona cadencia en forma de sus cinco eventos.

La inspección permite la adaptación. La inspección sin adaptación se considera inútil. Los eventos de Scrum están diseñados para provocar cambios.

Adaptación: si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse. El ajuste debe realizarse lo antes posible para minimizar la desviación adicional.

La adaptación se vuelve más difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para autogestionarse. Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

Valores de SCRUM

El uso exitoso de Scrum depende de que las personas sean más competentes en vivir cinco valores:

Compromiso, Enfoque, Apertura, Respeto y Coraje

El equipo de Scrum se compromete a lograr sus objetivos y apoyarse mutuamente. Su enfoque principal es el trabajo del Sprint para hacer el mejor progreso posible hacia estos objetivos. El equipo de Scrum y sus partes interesadas están abiertos sobre el trabajo y los desafíos. Los miembros del equipo de Scrum se respetan mutuamente para ser personas capaces e independientes, y son respetados como tales por las personas con las que trabajan. Los miembros del equipo de Scrum tienen el valor de hacer lo correcto y de trabajar en problemas complejos.

Scrum Team

El equipo de Scrum consta de un Scrum Master, un propietario de producto (Product Owner) y desarrolladores. Dentro de un equipo de Scrum no hay sub-equipos ni jerarquías. Es una unidad cohesiva de profesionales enfocada en un objetivo a la vez, el objetivo del producto.

Los equipos de Scrum son multifuncionales, lo que significa que los miembros tienen todas las habilidades necesarias para crear valor en cada Sprint. También son autogestionados, lo que significa que internamente deciden quién hace qué, cuándo y cómo.

El equipo de Scrum es lo suficientemente pequeño como para permanecer ágil y lo suficientemente grande como para completar un trabajo significativo dentro de un Sprint, por lo general 10 o menos personas. En general, hemos descubierto que los equipos más pequeños se comunican mejor y son más productivos.

El equipo Scrum es responsable de todas las actividades relacionadas con los productos, desde la colaboración, verificación, mantenimiento, operación, experimentación, investigación y desarrollo, y cualquier otra cosa que pueda ser necesaria. Están estructurados y empoderados por la organización para gestionar su propio trabajo.

Todo el equipo de Scrum es responsable de crear un incremento valioso y útil en cada Sprint. Scrum define tres responsabilidades específicas dentro del equipo de Scrum: los desarrolladores, el propietario del producto (Product Owner) y el Scrum Master.

Desarrolladores

Los desarrolladores son las personas del equipo de Scrum que se comprometen a crear cualquier aspecto de un incremento útil (funcional) en cada Sprint. Las habilidades específicas que necesitan los desarrolladores son a menudo amplias y variarán con el dominio del trabajo.

Son los responsables de:

- Crear un plan para el Sprint, el Sprint Backlog
- Inculcar la calidad adhiriéndose a una DoD
- Adaptar su plan cada día hacia el objetivo del Sprint
- Responsabilizarse mutuamente como profesionales

Product Owner

El Propietario del Producto es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. La forma en que esto se hace puede variar ampliamente entre organizaciones, equipos Scrum e individuos.

El Propietario del Producto también es responsable de la gestión eficaz de la pila del producto (Product Backlog), que incluye:

- Desarrollar y comunicar explícitamente el Objetivo del Producto.
- Creación y comunicación clara de elementos de trabajo pendiente del producto.
- Pedido de artículos de trabajo pendiente del producto.
- Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido.

Scrum Master

El Scrum Master es responsable de establecer Scrum tal como se define en la Guía de Scrum. Lo consigue ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo como en toda la organización.

El Scrum Master es responsable de la efectividad del Scrum Team. Lo logra al permitir que el equipo Scrum mejore sus prácticas, dentro del marco de Scrum. Los Scrum Masters son verdaderos líderes que sirven al equipo Scrum y a toda la organización.

El Scrum Master sirve al equipo de Scrum de varias maneras, incluyendo:

- Capacitar a los miembros del equipo en autogestión y multifuncionalidad.
- Ayudar al equipo de Scrum a centrarse en la creación de incrementos de alto valor que cumplan con DoD.
- Promover la eliminación de los impedimentos para el progreso del equipo Scrum.
- Asegurar de que todos los eventos de Scrum se llevan a cabo, sean positivos, productivos y que se respete el timebox.

Eventos de SCRUM

El **Sprint** es un contenedor para todos los eventos. Cada evento en scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de scrum.

Los **Sprints** están diseñados específicamente para permitir la transparencia necesaria. Si no se realizan los eventos según lo prescrito, se pierden oportunidades para inspeccionar y adaptarse.

Los **Sprints** se utilizan en scrum para crear regularidad y minimizar la necesidad de reuniones no definidas en scrum. De manera óptima, todos los eventos se llevan a cabo al mismo tiempo y lugar para reducir la complejidad.

Los **Sprints** son el latido del corazón de SCRUM, donde las ideas se convierten en valor.

Los **Sprints** son eventos de longitud fija de un mes o menos para crear consistencia. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior.

Todo el trabajo necesario para alcanzar el objetivo del producto, incluyendo la Planificación (Sprint Planning), Daily Scrums, Revisión del Sprint (Sprint Review) y la Retrospectiva (Sprint Retrospective), ocurren dentro del Sprints.

Durante el Sprint:

- No se hacen cambios que pongan en peligro el Objetivo Sprint.
- La calidad no disminuye.
- El trabajo pendiente del producto se refina según sea necesario.
- El alcance se puede clarificar y renegociar con el PO a medida que se aprende más.

Los Sprints permiten la previsibilidad al garantizar la inspección y adaptación del progreso hacia un objetivo del Producto, como mínimo, una vez al mes en el calendario. Los Sprints más cortos se pueden emplear para generar más ciclos de aprendizaje y limitar el riesgo de coste y esfuerzo a un período de tiempo más pequeño. Cada Sprint puede considerarse un proyecto corto.

1. Planificación de Sprint (Sprint Planning)

El Sprint Planning inicia el Sprint estableciendo el trabajo que se realizará en el mismo. Este plan resultante es creado por el trabajo colaborativo de todo el equipo de scrum.

Se discuten los elementos de trabajo pendiente de producto más importantes y cómo se asignan al objetivo de producto.

Se abordan los siguientes temas:

- * *¿Por qué el Sprint es valioso?* El PO propone cómo el producto podría aumentar su valor y utilidad en el Sprint actual. En equipo se define un objetivo de Sprint que comunique por qué el Sprint es valioso.
- * *¿Qué se puede hacer este Sprint?* Los desarrolladores seleccionan los elementos del Product Backlog para incluir en el Sprint actual. Selecciona cuánto se debe completar dentro de un Sprint.
- * *¿Cómo se realizará el trabajo elegido?* Para cada PBI seleccionado, los desarrolladores planifican el trabajo necesario para crear un incremento que cumpla con la definición de hecho.

Tiene una duración máxima de ocho horas para un Sprint de un mes.

2. Scrum Diario (Daily Scrum)

El propósito del Daily Scrum es inspeccionar el progreso hacia el Objetivo Sprint y adaptar el Sprint Backlog según sea necesario, ajustando el próximo trabajo planeado.

El Daily Scrum es un evento de 15 minutos máximo para los desarrolladores del equipo de Scrum. Para reducir la complejidad, se lleva a cabo al mismo tiempo y lugar todos los días laborables del Sprint.

Los desarrolladores pueden seleccionar cualquier estructura y técnicas que deseen, siempre y cuando su Scrum diario se centre en el proceso hacia el objetivo del Sprint y produzca un plan accionable para el día siguiente de trabajo. Crea enfoque y mejora la autogestión.

Mejoran la comunicación, identifican impedimentos, promueven una rápida para la toma de decisiones, y en consecuencia, eliminan la necesidad de otras reuniones.

3. Revisión del Sprint (Sprint Review)

El propósito es inspeccionar el resultado del Sprint y determinar futuras adaptaciones. El equipo de Scrum presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso hacia el objetivo de producto.

El equipo de Scrum y las partes interesadas revisan lo que se logró en el Sprint y lo que ha cambiado en su entorno. En base a esto, los asistentes colaboran en qué hacer a continuación. El trabajo pendiente del producto también se puede ajustar para satisfacer nuevas oportunidades.

Sprint Review es una sesión de trabajo y el equipo de Scrum debe evitar limitarla a que se convierta en una siempre presentación.

Máximo de cuatro horas para un Sprint de un mes.

4. La retrospectiva del Sprint (Sprint Retrospective)

El propósito es planificar formas de aumentar la calidad y la eficacia.

El equipo de Scrum inspecciona cómo fue el último Sprint con respecto a individuos, interacciones, procesos, herramientas y su definición de Hecho.

Las suposiciones que los desviaron se identifican y se exploran sus orígenes. El equipo de Scrum analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no fueron) resueltos.

El equipo de Scrum identifica los cambios más útiles para mejorar su eficacia. Las mejoras más impactantes se abordan lo antes posible. Incluso se pueden agregar al Sprint Backlog para el próximo Sprint.

Máximo de tres horas para un Sprint de un mes.

5. Refinamiento del Product Backlog

Agregar y refinar ítems. Mantener el Product Backlog actualizado, es una actividad continua y se usa cuando sea necesario. La llama el Product Owner.

Artefactos de Scrum

Los artefactos de Scrum representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave. Cada uno de los que inspecciona tienen la misma base para la adaptación.

Cada artefacto contiene un compromiso para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir progreso:

- * Trabajo pendiente del producto es el objetivo del producto.
- * El Sprint Backlog es el Sprint Goal.
- * Para el incremento es la Definición de Hecho.

Estos compromisos existen para reforzar el empirismo y los valores de Scrum para el equipo de Scrum y sus partes interesadas.

1. Pila del Producto (Product Backlog)

El trabajo pendiente del producto es una lista emergente y ordenada de lo que se necesita para mejorar el producto. Es la única fuente de trabajo emprendida por el equipo Scrum.

Los elementos de trabajo pendiente de producto se consideran listos para su selección en un evento de planificación de Sprint. Adquieren este grado de transparencia después de las actividades de refinación.

El refinamiento de Backlog de producto es el acto de descomponer y definir aún más los elementos de trabajo pendiente del producto en artículos más pequeños y precisos. Se agregan detalles, descripción, un pedido y un tamaño.

Compromiso: Objetivo del Producto (Product Goal)

El objetivo del producto describe un estado futuro del producto que puede servir como objeto para el equipo De Scrum contra el cual planificar. Se encuentra en el Product Backlog.

2. Pila del Sprint (Sprint Backlog)

El Trabajo pendiente de Sprint se compone del objetivo Sprint, el conjunto de elementos de trabajo pendiente de producto seleccionados para el Sprint (qué), así como un plan accionable para entregar el incremento (cómo).

Es un plan por y para los desarrolladores. Es una imagen del trabajo que los desarrolladores planean realizar durante el Sprint para lograr el objetivo Sprint. El Sprint Backlog se actualiza a lo largo del Sprint a medida que se aprende más.

Compromiso: Sprint Goal

Único objetivo para el Sprint. Es un compromiso de desarrolladores.

El objetivo de Sprint crea coherencia y enfoque, animando al equipo de Scrum a trabajar juntos en lugar de en iniciativas separadas.

Se crea durante el Sprint Planning y, a continuación, se agrega al Trabajo pendiente de Sprint. A medida que los desarrolladores trabajan durante el Sprint, tienen en cuenta el objetivo de Sprint.

3. Incremento (Increment)

Un incremento es un paso de hormigón hacia el Objetivo del Producto. Cada incremento es aditivo a todos los incrementos anteriores y verificado a fondo, asegurando que todos los incrementos funcionen juntos. Para proporcionar valor, el incremento debe ser utilizable.

Se pueden crear varios incrementos dentro de un Sprint. La suma de los Incrementos se presenta en la Revisión Sprint apoyando así el empirismo.

Compromiso: Definición de Hecho (Definition Of Done)

La Definición de Hecho es una descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto.

En el momento en que un elemento de trabajo pendiente de producto cumple con la definición de hecho, se crea un incremento.

La definición de Hecho crea transparencia al proporcionar a todos una comprensión compartida de qué trabajo se completó como parte del Incremento. Si un elemento de trabajo pendiente de producto no cumple con la definición de hecho, no se puede liberar, ni siquiera presentar en la revisión de Sprint. En su lugar, vuelve al Trabajo pendiente del producto para su consideración futura.

Timebox

Existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos de alcance del producto y no del tiempo o los costos (relacionado a la triple restricción).

Si no se llega al final de un sprint, en vez de extender el tiempo, se entrega menos.

Nos vemos más confiables con un ritmo de trabajo sostenible. Todas las ceremonias son Timebox. El refinamiento del Product Backlog al ser una actividad continua, no tiene definido un tiempo exacto, sino que el tiempo se expresa en términos porcentuales sobre la definición del Sprint.

Definición de Listo – DoR

El DoR es un criterio que define cuando una US está lo suficientemente bien formulada como para poderse incluir en un Sprint.

El DoD es un criterio que dice cuando una historia está lo suficientemente bien implementada como para entrar al Sprint Review y ser mostrada al PO, entonces el será el encargado de aprobar o no la US y en caso de que esté aprobada decidirá si desea ponerla en producción. Se arma un checklist con todo lo que debe cumplir una US para entrar en producción.

En el DoD se arma un checklist con todo lo que debe cumplir una US para entrar en producción.

Capacidad del Equipo en un Sprint

La capacidad es una métrica que utiliza SCRUM para determinar cuánta cantidad de trabajo puede comprometer un equipo para un determinado Sprint.

La capacidad se estima y se mide.

Una de las cosas que se hace en el Sprint Planning es determinar la capacidad del equipo, y entonces, teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.

Para equipos más maduros, la capacidad puede ser estimada en Story Points y para equipos menos maduros se puede estimar en horas ideales.

Niveles de Planificación

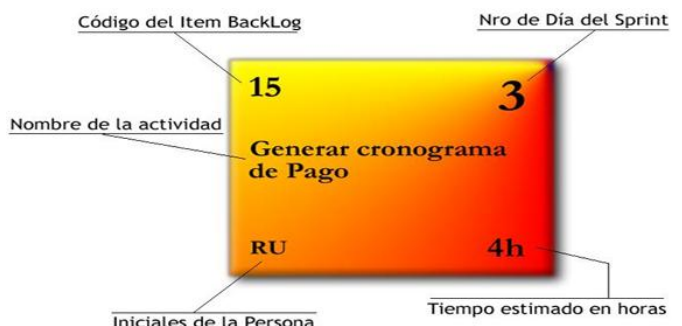
Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholder	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características
Release	3 (o menos) a 9 meses	Equipo Scrum entero y Stakeholders	Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de reléase
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente Sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de desarrollo	Cómo completar lo comprometido	Inspección del progreso

Herramientas de SCRUM

Tarjeta de tarea

Tarjeta que cuenta con cinco partes:

1. Código de Ítem Backlog
2. Nombre de la actividad: suele ser una frase verbal que define qué es lo que hay que hacer.
3. Iniciales del responsable de realizar la tarea.
4. Numero de día del Sprint.
5. Estimación del tiempo para finalizar la tarea.



Tablero

Tablero utilizado por el equipo de desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint.

Se divide en 5 sectores:

- * Story: nombre de la historia.
- * To Do: tareas por hacer en el Sprint.
- * Work in Process: tareas que se están realizando.
- * To Verify: tareas finalizadas que deben verificarse.
- * Done: tareas terminadas: finalizadas y verificadas.

Lo idea es que el nivel de granularidad de las tareas sea lo más fino posible, de tal forma que se detalle profundamente el avance del sprint, teniendo varias tareas en cada sección del tablero.

Gráficos del Backlog

Gráficos que brindan información acerca del progreso de un Sprint, de una release o del producto. El backlog de trabajo es la cantidad de trabajo que queda por ser realizado (en horas), mientras que la tendencia del backlog compara esta cantidad con el tiempo (medido en días).

Sprint Burn-Down Chart

Visualiza con una curva descendente cuánto trabajo queda por terminar. En el eje de las abscisas se coloca el tiempo que va desde el inicio del Sprint. En el eje de las ordenadas se coloca los puntos de historia a realizar en el Sprint.

Sprint Burn-Up Chart

Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir, que visualiza los puntos de historia terminados en cada Sprint.

Combined Burn Chart

Visualiza cuánto trabajo queda para terminar y cuánto trabajo se ha realizado.

UNIDAD N3 – GESTIÓN DE SOFTWARE COMO PRODUCTO

SCM – GESTIÓN DE CONFIGURACIÓN DE SOFTWARE

Software es un conjunto de programas y la documentación que lo acompaña. La idea del software como información o conocimiento empaquetado a distintos niveles de abstracción, donde el nivel más bajo es el código.

- * Información estructurada con propiedades lógicas y funcionales.
- * Información creada y mantenida en varias formas y representaciones.
- * Información confeccionada para ser procesada por computadora en su estado más desarrollado.

Los sistemas de software siempre cambian durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas versiones del sistema. Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Es necesario para no perder la trazabilidad de los cambios que se incorporan a cada versión.

Cambios en el Software

Tiene su origen en:

- * Cambios del negocio y nuevos requerimientos.
- * Soporte de cambios de productos asociados.
- * Reorganización de las prioridades de la empresa por crecimientos.
- * Cambios en el presupuesto.
- * Defectos encontrados a corregir.
- * Oportunidades de mejora.

Gestión de Configuraciones

La administración o gestión de configuraciones es una disciplina de soporte, forma parte de las disciplinas protectoras y tiene el propósito de mantener la integridad de los productos del proyecto de software a lo largo del ciclo de vida.

Es una disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, que surge para enfrentar la crisis de problemas de software, aplicando dirección y monitoreo (administrativo y técnico) a:

- * Identificar características técnicas y funcionales de ítems de configuración.
- * Documentar características técnicas y funcionales de ítems de configuración.
- * Controlar los cambios de esas características identificadas y documentadas.
- * Registrar y reportar estos cambios.
- * Verificar correspondencia con los requerimientos.

Su propósito es resolver problemas de diferente índole, a través del establecimiento y mantenimiento de la integridad de los productos de software a lo largo de su ciclo de vida. Involucra para la integridad del producto:

- * Pérdida de componentes.
- * Pérdida de cambios.
- * Regresión de fallas.
- * Doble mantenimiento.
- * Superposición de cambios.
- * Cambios no validados.

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

La integridad es el medio por el cual podemos garantizar que el producto a entregar tiene la calidad correspondiente. Y nos garantiza un nivel mínimo de confiabilidad.

La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.

Decimos que se mantiene la integridad de un producto de software cuando:

- * Satisface las necesidades del usuario: hace lo que el usuario espera que haga.
- * Permite la rastreabilidad durante todo su ciclo de vida: existen vínculos o conexiones entre los ítems de configuración, que permiten analizar en donde impactará un cambio en un ítem de configuración.
- * Satisface criterio de performance y RNF.
- * Cumple con sus expectativas de costo: incluye la satisfacción del equipo de proyecto. No solo el cliente debe estar satisfecho, sino también el desarrollo del producto debe ser redituable.

Ítem de Configuración

Se llama ítem de configuración a todos y cada uno de los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales necesitamos conocer su estado y evolución.

Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida (ya sea del producto o proyecto, dependiendo del artefacto).

Es decir, es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener.

Versión

Instancia de un ítem de configuración que difiere de otras instancias de este ítem.

Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado. Instancia de un ítem de configuración que difiere de otras instancias de este ítem.

Las versiones se identifican unívocamente. Controlar una versión refiere a la evolución de un único ítem de configuración. El control de versiones se refiere a la evolución de un único ítem de configuración (IC), o de cada IC por separado. La evolución puede representarse gráficamente en forma de grafo.

El control de versiones hace referencia a la evolución de un único ítem de configuración o de cada ítem de configuración por separado.

Variante

Una variante es una versión de un ítem de configuración (o de la configuración) que evoluciona por separado. Las variantes representan configuraciones alternativas.

Un producto de software puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale. Las variantes de un ítem de configuración permiten satisfacer requerimientos en conflicto, como por ejemplo la compatibilidad del producto con diferente hardware o sistemas operativos. Está ligado a trabajar con Branches.

Configuración del Software

La configuración del software es un conjunto de ítems de configuración con su correspondiente versión en un momento determinado. Define una foto de los ítems de configuración en un momento de tiempo determinado.

La configuración de un software es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.

Repositorio

Es un contenedor de ítems de configuración, se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos.

Tiene una estructura para mantener el orden y la integridad. El que tenga una estructura ayuda a la seguridad, los controles de acceso, las políticas de Backup y todo aquello que se aplica sobre un repositorio.

1. **Repositorios Centralizados:** Un servidor contiene todos los archivos con sus versiones. Los administradores tienen mayor control sobre el repositorio. Tiene como desventaja que si falla el servidor, todo lo positivo se cae.
2. **Repositorios Descentralizados:** Cada cliente tiene una copia exactamente igual del repositorio completo. Si un servidor falla, sólo es cuestión de "copiar y pega". Posibilita otros workflows no disponibles en el modelo centralizado.

Release

Entrega de un sistema que se libera para su uso a los clientes u otros usuarios de la organización.

Rama

Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos: experimentación, resolución de errores en el desarrollo y para desarrollar un mismo software para diferentes plataformas.

La integración de ramas se da cuando se integran dos ramas, para fusionar la configuración de los ítems de configuración con sus correspondientes versiones, en cada una de ellas.

Línea Base

Una línea base es un conjunto de ítems de configuración que han sido construidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores.

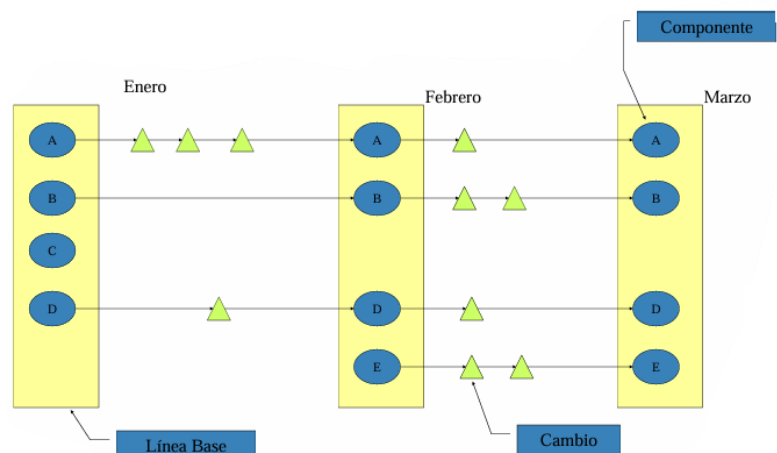
Una especificación o producto que se ha revisado formalmente y sobre los que se ha llegado a un acuerdo, y que, de ahí en adelante, sirve como base para un desarrollo posterior y que puede cambiarse solamente a través de procedimientos formales de control de cambio.

La línea base es una configuración de software que ha sido formalmente revisada y aprobada, que sirve como base para desarrollos futuros. Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.

Se utilizan etiquetas para "marcar" las base line, debe tener una referencia única, y esto se hace a través de "tags".

La línea base es una configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo. Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios.

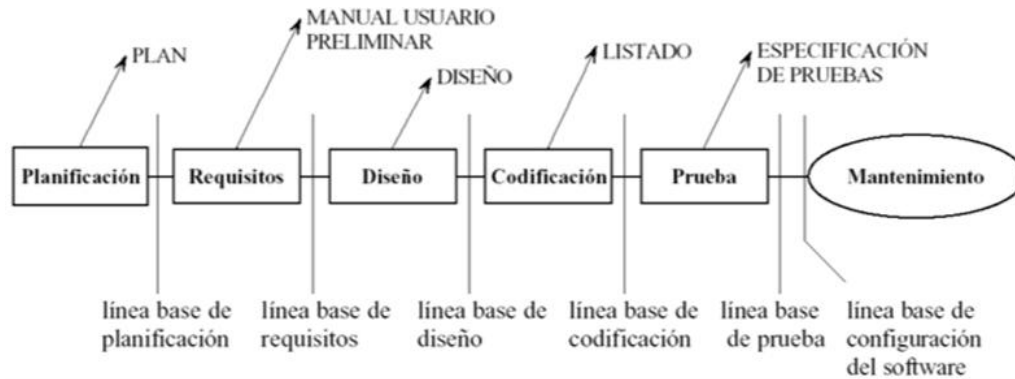
Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.



Representación de Líneas Base

Pueden ser:

- * **De especificación (requerimientos, diseño):** son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimientos.
- * **Operacionales:** contiene una versión de producto cuyo código es ejecutables, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release. Línea base de productos que han pasado por un control de calidad definido previamente.



Se logra mediante la definición de sus líneas base y de los cambios que los mismos puede sufrir durante la evolución del sistema.

Gestión de Configuración de Software

Una disciplina que aplica dirección y monitoreo administrativo y técnico a: identificar y documentar las características funcionales y técnicas de los ítems de configuración, controlar los cambios de esas características, registrar y reportar los cambios y su estado de implementación y verificar correspondencia con los requerimientos.

Actividades Fundamentales

Estas actividades/elementos son las cosas que contienen las secciones de un plan de gestión de configuración. Recordemos que la gestión de configuración también se debe planificar.

1. Identificación de los Ítems de Configuración

Cuando se desea controlar los cambios de un elemento es necesario identificarlo, para lo que se declara un ítem de configuración (SCI). Para que un ítem de configuración se vuelva tal debe estar versionado. Un ítem de configuración inicia en un estado borrador donde se van haciendo los cambios, luego pasa a una etapa de revisión y finalmente queda aceptado. Aquí queda en estado “baseline”, es decir, que no se puede cambiar si no es a través de un procedimiento formal de cambio.



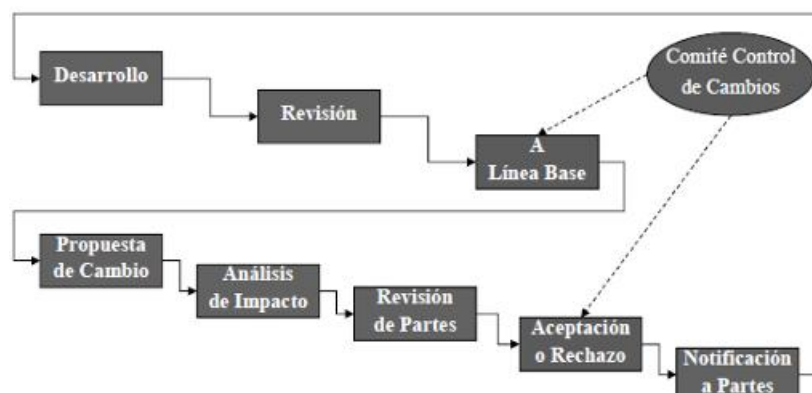
2. Control de Cambios

Una vez que la línea base se conformó, no es posible cambiarla sin pasar por un proceso formal, llevado a cabo por lo que se conoce como comité de control de cambios. La formalidad del proceso está dada por el hecho de que todos los involucrados se anoticien.

Esta autoridad de cambio, al recibir una "propuesta de cambio", lleva adelante un análisis de impacto del cambio, donde se evalúa el esfuerzo técnico, efectos secundarios, impacto global sobre otras funciones y sobre otros objetos, se le asigna una prioridad para que posteriormente se realice lo que se conoce como revisión de partes, en base a todo este análisis el comité acepta o rechaza el cambio y notifica a todas las partes involucradas.

Comité de Control de Cambios:

- * Involucrados directos que están en el proyecto.
- * Si el cambio se origina en el cliente, se lo incluye.
- * La convocatoria se realiza para analizar el cambio sobre una línea base, y no un ítem en particular



3. Auditorías de Gestión de Configuración

La auditoría de configuración tiene como objetivo asegurar que lo que está indicado para cada Ítem de Configuración de Software en la línea base o actualización se ha alcanzado realmente y que el software y la documentación son internamente consistentes para entregarlos al cliente.

Debe ser objetiva e independiente. El que controla no debe estar involucrado con lo controlado, debe ser externo del proyecto.

Auditoría física de configuración (PCA): asegura que lo que está indicado para cada ICS en la línea base o en la actualización se ha alcanzado realmente.

Auditoría funcional de configuración (FCA): evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos.

Sirve para dos procesos básicos: la validación y la verificación.

Validación: el problema es resuelto de manera apropiada que el usuario obtenga el producto correcto.

Verificación: asegura que un producto cumple con los objetivos preestablecidos, definidos en la documentación de líneas base. Todas las funcionalidades son llevadas a cabo con éxito y los test cases tengan status "ok" o bien consten como "problemas reportados" en la nota de release.

4. Registro e Informes de Estado

Se ocupan de mantener los registros de la evolución del sistema. Maneja mucha información y salidas por lo que suele implementar dentro de procesos automáticos. Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida.

Plan de Gestión de Configuración

Se hace en el enfoque tradicional. El plan debería incluir:

- * Reglas de nombrado de los CI.
- * Herramientas a utilizar para SCM.
- * Roles e integrantes del Comité.
- * Procedimiento formal de cambios.
- * Plantillas de formularios.
- * Procesos de auditoría.

Gestión de configuración en ambientes ágiles

En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa.

En orden con la caracterización de los equipos ágiles, decimos que la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.

Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles. Todos los procesos definidos establecidos en la gestión de configuración del enfoque tradicional son relativizados en agile. Por ejemplo, no existe un comité para el proceso forma de control de cambios.

Podemos decir que va de la mano con el manifiesto ágil porque recordemos que se buscaba estar siempre preparados para el cambio y justamente, la gestión de configuración busca responder a los cambios y mantener la integridad del producto.

La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto. Es decir, trasciende el proyecto.

Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto.

No Silver Bullet – Fred Brooks

Contrastar progreso del hardware (algo tangible, construible, que se puede mejorar en su construcción), en comparación con el software (intangible, complejo inherentemente).

Las balas de plata son aquellas soluciones que podrían hacer frente a la crisis del software: no existe un desarrollo de software que demuestre avances de progresión (simplicidad, fiabilidad y productividad) en la construcción del software en menos de una década.

Dificultades que plantea: el software tiene complejidad esencial (del dominio) y accidental (de las herramientas)

- * **Esenciales:** todo aquello que trae aparejado el construir software: conceptos abstractos, estructuras de datos, algoritmos, invocación de funciones, etc. (carece de precisión y detalles). 4 aspectos la caracterizan:
 - √ Complejidad:
 - propiedad inherente, debido a la abstracción del software
 - al crecer el tamaño del software, su complejidad crece de manera exponencial, no lineal
 - Esto conduce a muchos problemas: dificultad de comunicación entre miembros de equipos, deficiencias en el producto, errores en cumplimiento de fechas, excesos en costos, etc.
 - √ Conformidad:
 - el software debe realizar aquello que el cliente necesita, esto es: debe adecuarse a sus requerimientos.
 - √ Mutabilidad:
 - el software está sujeto a constantes cambios. A diferencia de objetos tangibles, el software muta y sufre cambios a lo largo del tiempo. En cambio, en muchos objetos esto no es así, ya que se reemplazan por nuevos productos (no tomo el objeto y lo modifico, creo uno nuevo).
 - √ Invisibilidad:
 - al ser algo no visualizable, se dificulta mucho trabajar con representaciones geométricas, como puede ser con estructuras mecánicas. Los grafos son una buena herramienta que permiten modelar muchas situaciones de un software.
- * **Accidentales:** tiene que ver con aquellos aspectos que dificultan la construcción del software, pero han sido solucionados a lo largo del tiempo. Esas soluciones son:
 - √ Lenguajes de programación de alto nivel (low-code): permiten abstraer el proceso de construcción del software, independizándose del hardware en el que se ejecutarán. Esto elimina la complejidad propia de una máquina, que nada tiene que ver con el software que se desea construir.
 - √ Tiempo compartido: reducción de los tiempos de respuesta de un software. Esto permite no desviar el foco de lo que se desea construir.
 - √ Entornos de programación unificados: facilitan la construcción del software al integrar librerías, formato de archivos, frameworks, etc.

Resolución de dificultades accidentales

- * Programación orientada a objetos: permite a los diseñadores del software, abstraerse de tecnicismos para expresar su diseño simplemente en un lenguaje de máquina. Esto facilita el diseño, pero sólo elimina una dificultad accidental, no elimina la propia dificultad esencial de diseñar software.
- * Inteligencia Artificial: si bien la IA permite resolver muchas cuestiones que reemplazan acciones humanas, siempre tiene que haber un equipo de personas que programe y “enseñe” a esa IA cómo actuar. Con lo cual, el factor humano no se reemplaza del todo.
- * Sistemas expertos: son sistemas que aplican la IA y permiten sugerir diferentes situaciones como estrategias de Testing, optimizaciones de código, sugerencias en nombrado de variables, etc. Igualmente, no se quita el factor humano ya que se necesita encontrar expertos y desarrollar técnicas eficientes para extraer lo que ellos saben para destilarlo dentro de bases de reglas de estos sistemas. El prerrequisito esencial para construir un sistema experto es tener un experto

- * Programación automática: esto se planteó como un reemplazo a los programadores, pero en lugar de ser eso, terminan siendo un lenguaje de más alto nivel que el programador debe parametrizar.
- * Programación gráfica
 - ✓ Permite programar a un nivel más alto de abstracción que la programación tradicional, pero sigue estando el factor humano presente, con lo que solo se logra dar una mayor abstracción al programador sin que este tenga la necesidad de aprender sintaxis del lenguaje.
- * Verificación de programas:
 - ✓ Lo que se logra con la verificación de programa, es establecer que un programa pasa las pruebas matemáticas de verificación que se proveen.
 - ✓ No permite verificar que se cumplan todas las necesidades del cliente que fueron previstas en el software en forma de funcionalidades, sino que verifica que las funcionalidades desarrolladas no tengan errores.
 - ✓ Además, las pruebas matemáticas pueden también estar sometidas a errores.
- * Environments and tools
 - ✓ La mayoría de los nuevos entornos inteligentes tan solo nos permiten librarnos de errores semánticos y sintácticos simples.
- * Estaciones de trabajo poderosas
 - ✓ El incremento de potencia y memoria de las estaciones de trabajo individuales no soluciona el déficit en el desarrollo del software.
 - ✓ La edición de programas y documentos son soportadas totalmente por las máquinas de hoy en día.
 - ✓ Una mejora en tiempos de compilación no representa un avance significativo.

Resolución de dificultades esenciales

- * Comprar, antes que construir: explorar en el mercado soluciones que se puedan conseguir para reemplazar lo que se quiera construir. Si existe un mercado que lo ha demandado, comprarlo siempre será más barato que construirlo. Esto se debe a que al tener una gran cantidad de clientes que lo compren, el costo de ese software se amortiza. Mirarlo como que el uso de N copias de software, multiplica la productividad de ese software N veces. Lógicamente este enfoque es difícil de utilizar para necesidades muy puntuales, donde no existe una porción de mercado que lo haya demandado previamente.
- * Usar prototipos y refinar requerimientos: la parte fundamental de construir software es saber qué se debe construir. La extracción iterativa de requerimientos y su posterior refinamiento permiten tomar las necesidades de los clientes en sucesivas iteraciones, debido a que ellos mismos no saben expresar ni conocer muchas veces cuáles son esas necesidades que tienen. Además, es prácticamente imposible que sepan expresar con detalle y precisión todos sus requerimientos para el software que necesitan. El desarrollo de prototipos rápidos permite cubrir esa extracción iterativa de requerimientos, al simular interfaces y funciones principales de un software que se necesita construir. Esto permite obtener un feedback valioso del cliente, para que todos comprendan mejor cuáles son las funcionalidades que se necesitan, y cómo se deben ejecutar.
- * Desarrollo incremental: (crecer y no construir software): Incrementar el software añadiendo funcionalidades mientras estas se prueban, usan y testean.

Tener un software con pocas funcionalidades, pero funcionando, es más animador que tener un avance de un software complejo que no funciona.

- * Personas capacitadas: que sigan buenas prácticas. Esto permite tener un buen equipo, pero, al fin y al cabo, desarrollar software requiere de creatividad, por lo que priorizar la creatividad en un equipo puede traducirse en grandes resultados (estructuras rápidas, pequeñas, simples construidas con menos esfuerzo).

Esto plantea que tener en un equipo con buenos diseñadores, es tan importante como tener buenos líderes que dirijan ese equipo. Por lo que se debe recompensarlos de la misma forma, debido a su gran valor para el desarrollo del software.

Resumen: La dificultad del software es principalmente conceptual, no técnica. Las herramientas reducen la complejidad accidental, pero nada elimina la complejidad esencial del problema que estamos modelando. Ninguna herramienta puede reemplazar la experiencia.